



**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN**

**ĐỒ ÁN MÔN HỌC
THỊ GIÁC MÁY TÍNH**

**TRÍCH XUẤT THÔNG TIN
BIÊN LAI TIẾNG VIỆT**

*So sánh kiến trúc Donut (OCR-free)
và VietOCR + LayoutXLM (OCR-based)*

Giảng viên hướng dẫn:

ThS. Nguyễn Đình Quý

Bộ môn:

Khoa học máy tính

Sinh viên thực hiện:

Trần Phùng Đức Anh - 0204168

Nguyễn Minh Năng - 0211668

Nguyễn Việt Hùng - 0208768

Lớp:

68CS2

Hà Nội, 2026

Lời cảm ơn

Lời đầu tiên, chúng em xin bày tỏ lòng biết ơn sâu sắc tới ThS. Nguyễn Đình Quý, giảng viên bộ môn Khoa học máy tính, Khoa Công nghệ thông tin, Trường Đại học Xây dựng Hà Nội. Thầy đã tận tình hướng dẫn, định hướng khoa học và cung cấp những chỉ dẫn vô cùng quý báu trong suốt quá trình nghiên cứu và hoàn thành đồ án môn học Thị giác máy tính này.

Chúng em cũng xin gửi lời cảm ơn đến các thầy cô giáo trong Khoa Công nghệ thông tin đã truyền đạt những kiến thức nền tảng bổ ích, tạo điều kiện thuận lợi nhất để chúng em thực hiện đề tài.

Cuối cùng, xin cảm ơn gia đình, bạn bè và tập thể lớp 68CS2 đã luôn động viên, hỗ trợ và đóng góp ý kiến để nhóm hoàn thiện báo cáo đồ án này một cách tốt nhất. Do kiến thức và kinh nghiệm thực tế còn hạn chế, báo cáo không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự đóng góp và chỉ bảo của quý thầy cô.

Chúng em xin chân thành cảm ơn!

Hà Nội, năm 2026

Nhóm sinh viên thực hiện

Mục lục

Lời cảm ơn	1
Danh mục hình ảnh	5
Danh mục bảng biểu	6
Danh mục từ viết tắt	7
Tóm tắt	8
1 Tổng quan đề tài	9
1.1 Đặt vấn đề (Problem Statement)	9
1.2 Câu hỏi nghiên cứu (Research Questions)	9
1.3 Mục tiêu và chỉ số đo lường	10
1.4 Phạm vi đề tài	10
1.5 Đóng góp thực tiễn	11
1.6 Cấu trúc báo cáo	11
2 Cơ sở lý thuyết	12
2.1 Bài toán trích xuất thông tin tài liệu	12
2.2 Các công trình nghiên cứu liên quan	12
2.3 Nhận dạng ký tự quang học (OCR)	14
2.3.1 Phát hiện văn bản với PaddleOCR	14
2.3.2 Nhận dạng văn bản tiếng Việt với VietOCR	14
2.4 Mô hình Donut (OCR-free)	14
2.4.1 Kiến trúc tổng quan	15
2.4.2 Phân biệt khái niệm Warm-up trong Donut	15
2.5 Mô hình LayoutXLM (OCR-based)	16
2.5.1 Nhúng đa phương thức (Multimodal Embedding)	16
2.5.2 Spatial-aware Self-Attention	16
2.5.3 Gán nhãn BIO (BIO Tagging)	17
2.6 Các độ đo đánh giá và quy ước xử lý rỗng	17
2.6.1 Exact Match (EM)	17
2.6.2 Normalized Edit Similarity (NES)	17
2.6.3 Character Error Rate (CER)	18
2.6.4 Hai chế độ tổng hợp kết quả	18

3	Tập dữ liệu	19
3.1	Nguồn dữ liệu	19
3.2	Schema JSONL thống nhất	19
3.3	Chia dữ liệu và chống rò rỉ	19
3.4	Mức annotation và dữ liệu huấn luyện	20
3.5	Giá trị ground-truth rỗng trên test	20
3.6	Kiểm tra hợp lệ	20
4	Tiền xử lý dữ liệu	21
4.1	Tiền xử lý ảnh (Image Preprocessing)	21
4.2	Tăng cường dữ liệu (Data Augmentation)	21
4.3	Chuẩn hoá văn bản (Text Normalization)	22
4.4	Xây dựng OCR Cache	22
4.5	Trạng thái khảo sát tiền xử lý OCR	23
5	Phương pháp thực hiện	24
5.1	Tổng quan kiến trúc hệ thống	24
5.2	Baseline: OCR + Trích xuất dựa trên luật (Rule-based)	24
5.2.1	Pipeline tổng quan	24
5.2.2	Quy tắc trích xuất cụ thể	24
5.3	Donut: End-to-End OCR-free	27
5.3.1	Chuẩn bị dữ liệu	27
5.3.2	Cấu hình huấn luyện	27
5.3.3	Suy luận (Inference)	28
5.4	VietOCR + LayoutXLM: OCR-based Layout-aware	28
5.4.1	Chuẩn bị dữ liệu	29
5.4.2	Cấu hình huấn luyện	29
5.4.3	Suy luận và hậu xử lý (Inference & Post-processing)	29
5.5	Hậu xử lý chung (Post-processing)	31
6	Thiết lập thực nghiệm	32
6.1	Trạng thái prediction artifact	32
6.2	Môi trường và cấu hình	32
6.3	Phạm vi dữ liệu của từng pipeline	32
6.4	Quy trình suy luận và đánh giá	33
6.5	Phạm vi phép đo latency	33
6.6	Trạng thái bằng chứng huấn luyện	34
7	Kết quả thực nghiệm và đánh giá	35
7.1	Kết quả trên toàn bộ mẫu và trên trường có mặt	35

7.2	Phân tích theo trường	36
7.3	Latency của các thành phần đã ghi nhận	37
7.4	Chẩn đoán artifact Donut	38
7.5	Phân tích lỗi có thể tái lập	38
7.6	Oracle OCR và câu hỏi về ảnh hưởng của OCR	39
7.7	Demo ứng dụng	39
8	Kết luận và hướng phát triển	40
8.1	Trả lời câu hỏi nghiên cứu	40
8.2	Đe dọa tính hợp lệ	40
8.3	Đóng góp và hạn chế	41
8.4	Hướng phát triển	41
	Tài liệu tham khảo	43
A	Mẫu dữ liệu JSONL thống nhất	44
B	Cấu trúc mã nguồn	45
C	Cấu hình huấn luyện đầy đủ	46
C.1	Cấu hình DONUT (donut.yaml)	46
C.2	Cấu hình LAYOUTXLM (layoutxlm.yaml)	47
C.3	Cấu hình luật heuristic (baseline.yaml)	48
D	Mẫu kết quả trích xuất thực tế	50
E	Giao diện Demo Gradio	51
E.1	Hướng dẫn cài đặt và chạy demo	51

Danh sách hình vẽ

2.1	Kiến trúc tổng quan mô hình DONUT (Swin Encoder kết hợp mBART Decoder) [4].	15
5.1	Sơ đồ tổng quan kiến trúc ba pipeline trích xuất thông tin biên lai.	25
5.2	Ví dụ gán nhãn Begin-Inside-Outside — Lược đồ gán nhãn chuỗi (BIO) cho các từ trên biên lai.	30
7.1	So sánh Exact Match trên các mẫu có ground-truth khác rỗng. Hình được sinh từ metrics JSON.	37
E.1	Giao diện Gradio trong một lượt chạy thực tế, so sánh kết quả của Baseline, DONUT và VietOCR + LAYOUTXLM.	51

Danh sách bảng

1.1	Mô tả các trường thông tin trong phạm vi trích xuất.	10
2.1	So sánh đặc tính và giả thuyết kiểm chứng của ba hướng tiếp cận.	13
2.2	Hệ thống nhãn BIO cho 4 trường thông tin trích xuất.	17
3.1	Các trường chính trong schema JSONL.	19
3.2	Phân bố mẫu trong các tệp processed.	20
3.3	Mức annotation theo split.	20
3.4	Số mẫu có ground-truth rỗng trên 234 mẫu test.	20
4.1	Các phép tăng cường dữ liệu được cài đặt cho hai mô hình.	21
4.2	Ví dụ chuẩn hoá văn bản theo trường thông tin.	22
5.1	Quy tắc trích xuất cho từng trường thông tin trong pipeline baseline.	26
5.2	Cấu hình huấn luyện mô hình DONUT.	28
5.3	Cấu hình huấn luyện mô hình LAYOUTXLM.	30
6.1	Trạng thái bằng chứng của các prediction artifact đã khóa bằng SHA-256.	32
6.2	Cấu hình huấn luyện được khai báo cho Donut và LayoutXLM.	33
6.3	Số mẫu có thể sử dụng ở từng giai đoạn.	33
7.1	Kết quả trên toàn bộ 234 mẫu kiểm thử của các artifact hợp lệ. Macro là trung bình không trọng số của bốn trường.	35
7.2	Kết quả present-only của các artifact hợp lệ: mỗi trường chỉ được chấm trên mẫu có ground-truth khác rỗng.	35
7.3	Độ phủ prediction và số mẫu đánh giá. Số mẫu present-only lần lượt là 224, 216, 217 và 223 cho bốn trường.	36
7.4	Paired bootstrap present-only cho chênh lệch LayoutXLM trừ Baseline, seed 20260620, 50,000 lần lấy mẫu.	36
7.5	Phân tích độ nhạy present-only sau khi loại bốn mẫu có sai lệch hệ tọa độ ảnh/OCR ($n = 230$).	36
7.6	Latency trung bình của các thành phần hợp lệ đã ghi nhận. Hai phép đo đều bắt đầu sau OCR cache và không đại diện cho latency end-to-end từ ảnh thô.	37
7.7	Artifact Donut chỉ dùng cho chẩn đoán, không dùng để xếp hạng.	38
7.8	Taxonomy lỗi được sinh tự động từ prediction artifact.	38
D.1	Ví dụ kết quả trích xuất thực tế so với nhãn ground-truth trên tập test.	50

Danh mục từ viết tắt

BART Bidirectional and Auto-Regressive Transformers. [27](#), [30](#)

BIO Begin-Inside-Outside — Lược đồ gán nhãn chuỗi. [6](#), [7](#), [14](#), [17](#), [18](#), [27](#), [31–34](#), [40](#)

IoU Intersection over Union — Tỷ lệ giao trên hợp. [32](#)

MC-OCR Mobile-Captured OCR — Nhận dạng ký tự chụp di động. [20](#)

NFC Unicode Normalization Form Composed — Dạng chuẩn hoá Unicode tổ hợp. [34](#), [40](#)

OCR Optical Character Recognition — Nhận dạng ký tự quang học. [10](#), [13](#), [27](#), [30–32](#), [40](#)

Tóm tắt

Đồ án xây dựng hệ thống trích xuất tên cửa hàng, ngày giao dịch, tổng tiền và địa chỉ từ ảnh biên lai tiếng Việt. Ba phương pháp được triển khai gồm Baseline dùng OCR và luật, DONUT sinh chuỗi trực tiếp từ ảnh, và LAYOUTXLM phân loại token từ văn bản, bố cục và ảnh. Bộ dữ liệu processed gồm 1.559 mẫu; Donut có thể sử dụng toàn bộ 1.091 mẫu train, còn LayoutXLM sử dụng 784 mẫu train có hộp bao. Các prediction artifact đều bao phủ cùng 234 ID test.

Để tránh điểm số bị ảnh hưởng bởi nhãn rỗng, báo cáo trình bày cả kết quả trên toàn bộ mẫu và kết quả present-only, chỉ xét các mẫu có ground-truth khác rỗng của từng trường. Trong ba phương pháp so sánh chính, LAYOUTXLM đạt kết quả tốt nhất với Macro EM 42,96% và Macro NES 63,89% trên present-only; Baseline đạt tương ứng 40,41% và 55,47%; trong khi DONUT (đã sửa lỗi cấu hình sinh 20 token) đạt hiệu năng rất thấp (Macro EM 0,00%) do hiện tượng lặp từ và suy sụp mô hình (model collapse) khi huấn luyện end-to-end trên tập dữ liệu nhỏ. Thí nghiệm Oracle OCR cho thấy khi có OCR hoàn hảo, hiệu năng LayoutXLM tăng vọt lên Macro EM 66,97% trên present-only, chỉ ra OCR là nút thắt cổ chai lớn nhất của pipeline. Báo cáo đồng thời cập nhật đầy đủ log huấn luyện, phân tích độ nhạy hệ tọa độ và hoàn thiện bằng chứng thực nghiệm của đồ án.

Từ khoá: trích xuất thông tin, biên lai, Donut, LayoutXLM, OCR, tiếng Việt.

1. Tổng quan đề tài

Chương này khái quát về đề tài nghiên cứu, phân tích tính cấp thiết của bài toán tự động hóa xử lý tài liệu tại Việt Nam. Nội dung tập trung làm rõ các câu hỏi nghiên cứu cốt lõi, mục tiêu đo lường, phạm vi hoạt động của hệ thống, cùng các đóng góp thực tiễn mà đề án đạt được.

1.1. Đặt vấn đề (Problem Statement)

Trong xu thế số hóa và xây dựng nền kinh tế số tại Việt Nam, nhu cầu tự động hóa nhập liệu tài liệu tài chính như biên lai, hóa đơn bán lẻ phát sinh vô cùng lớn tại các hệ thống siêu thị, cửa hàng tiện lợi, và doanh nghiệp dịch vụ. Việc nhập liệu và đối soát thủ công không chỉ tiêu tốn nhiều thời gian, chi phí vận hành mà còn tiềm ẩn nguy cơ sai sót do con người. Việc tự động hóa quy trình này bằng kỹ thuật nhận dạng ký tự quang học ([Optical Character Recognition — Nhận dạng ký tự quang học \(OCR\)](#)) kết hợp trích xuất thông tin then chốt (Key Information Extraction - KIE) là một đòi hỏi cấp thiết trong thực tiễn.

Tuy nhiên, bài toán trích xuất thông tin biên lai tiếng Việt trong môi trường thực tế đối mặt với nhiều thách thức lớn:

1. **Chất lượng ảnh không đồng đều:** Ảnh biên lai chụp bằng thiết bị di động cá nhân thường bị mờ, nghiêng, thiếu sáng, bị lóa phản quang, hoặc nhàu nát.
2. **Cấu trúc bố cục tự do:** Không có tiêu chuẩn chung về cách trình bày thông tin trên biên lai bán lẻ. Cấu trúc dòng chữ, bảng biểu biến động mạnh giữa các cửa hàng.
3. **Đặc trưng tiếng Việt có dấu:** Tiếng Việt sở hữu hệ thống dấu thanh và ký tự phức tạp. Sai lệch một dấu nhỏ trong quá trình nhận dạng có thể dẫn đến việc sai lệch toàn bộ nghĩa của từ (ví dụ nhầm lẫn tên cửa hàng hoặc thông tin địa chỉ).

Để giải quyết bài toán này, các hướng nghiên cứu hiện đại chia làm hai trường phái kỹ thuật chính:

- **OCR-based pipeline** (như kết hợp PaddleOCR/VietOCR với mô hình ngôn ngữ hiểu bố cục LAYOUTXLM [11]): Phân tách rõ ràng giai đoạn nhận dạng chữ viết và giai đoạn phân loại thực thể cấp token.
- **OCR-free pipeline** (như mô hình sinh end-to-end DONUT [4]): Sử dụng mạng nơ-ron Transformer trực tiếp sinh ra chuỗi có cấu trúc từ ảnh đầu vào mà không cần module nhận dạng văn bản trung gian.

Việc đánh giá hai hướng tiếp cận trên cùng tập kiểm thử giúp mô tả ưu, nhược điểm của các checkpoint hiện có. Tuy nhiên, một phép so sánh kiến trúc tuyệt đối còn đòi hỏi kiểm soát lượng dữ liệu và mức annotation giống nhau.

1.2. Câu hỏi nghiên cứu (Research Questions)

Để định hướng các nội dung thực nghiệm và đánh giá, đề án đặt ra ba câu hỏi nghiên cứu cốt lõi sau:

- **RQ1:** Trên các prediction artifact đủ điều kiện đánh giá, mô hình OCR-based LAYOUTXLM cải thiện như thế nào so với Baseline dựa trên luật? Artifact hiện có cho phép kết luận đến đâu về DONUT?
- **RQ2:** Các thành phần suy luận đã được ghi nhận có latency khác nhau như thế nào, và phép đo hiện tại cho phép kết luận đến đâu?
- **RQ3:** Dữ liệu hiện có cung cấp bằng chứng nào về sự phụ thuộc của pipeline OCR-based vào OCR, và cần thí nghiệm nào để định lượng ảnh hưởng đó?

1.3. Mục tiêu và chỉ số đo lường

Mục tiêu của đồ án là thiết kế, huấn luyện và đánh giá khách quan ba phương pháp trích xuất 4 trường thông tin quan trọng từ biên lai tiếng Việt: tên cửa hàng (**store_name**), ngày tháng (**date**), tổng tiền (**total**), và địa chỉ (**address**).

Để trả lời các câu hỏi nghiên cứu, hiệu năng của các phương pháp được đo lường định lượng trên cùng tập kiểm thử độc lập thông qua bốn chỉ số:

- **Exact Match (EM):** Tỷ lệ dự đoán khớp chính xác tuyệt đối từng ký tự với nhãn thật sau chuẩn hóa.
- **Normalized Edit Similarity (NES):** Đo độ tương đồng chuỗi ký tự dựa trên khoảng cách Levenshtein chuẩn hóa.
- **Character Error Rate (CER):** Tỷ lệ lỗi ký tự, đo lường chi tiết sai lệch chữ viết.
- **Latency thành phần:** Thời gian được ghi trong prediction artifact; phạm vi bắt đầu khác nhau giữa pipeline OCR-based và Donut.

1.4. Phạm vi đề tài

Đề tài tập trung tối ưu hóa và đánh giá khả năng trích xuất thông tin trên 4 trường dữ liệu cốt lõi được mô tả trong [Bảng 1.1](#).

Bảng 1.1: Mô tả các trường thông tin trong phạm vi trích xuất.

Trường	Định dạng chuẩn hóa	Ví dụ thực tế
store_name	Chuỗi Unicode NFC	CIRCLE K
date	Chuỗi định dạng YYYY-MM-DD	2021-03-15
total	Chuỗi số nguyên không dấu	115000
address	Chuỗi Unicode NFC	123 Nguyễn Trãi, Thanh Xuân

Ngoài phạm vi thực hiện (Out of Scope):

- Hệ thống không trích xuất danh sách chi tiết các mặt hàng mua sắm (items list).
- Không trích xuất mã số thuế (MST), số điện thoại, email, website hoặc số hóa đơn.
- Chỉ làm việc trực tiếp trên dữ liệu ảnh chụp (JPG, PNG), không xử lý file PDF số hóa.

1.5. Đóng góp thực tiễn

Đồ án mang lại các đóng góp thực tiễn cụ thể phục vụ bài toán trích xuất tài liệu tiếng Việt:

1. **Xây dựng bộ dữ liệu nhãn sạch:** Hợp nhất và gán nhãn chuẩn hóa tập dữ liệu gồm 1.559 mẫu biên lai tiếng Việt từ cuộc thi MC-OCR 2021 và dữ liệu tự chụp thực tế.
2. **Đánh giá có kiểm soát bằng chứng:** So sánh Baseline và LAYOUTXLM trên cùng tập kiểm thử, đồng thời tách artifact DONUT không hoàn chỉnh sang phạm vi chẩn đoán thay vì dùng để xếp hạng.
3. **Phân loại và phân tích lỗi:** Xây dựng taxonomy bảy nhóm lỗi có thể sinh lại tự động từ prediction, ground-truth và OCR cache.
4. **Demo ứng dụng tương tác:** Triển khai giao diện demo bằng Gradio chạy thời gian thực phục vụ mục đích kiểm thử trực quan.

1.6. Cấu trúc báo cáo

Báo cáo được tổ chức thành các chương cụ thể như sau:

- **Mục 2 (Cơ sở lý thuyết):** Trình bày bài toán trích xuất thông tin, lý thuyết về OCR, kiến trúc Donut, LayoutXLM và các công trình liên quan.
- **Mục 3 (Thu thập và xây dựng tập dữ liệu):** Mô tả các nguồn dữ liệu, quy tắc gán nhãn, schema thống nhất và quy trình loại mẫu trùng lặp.
- **Mục 4 (Tiền xử lý dữ liệu):** Chi tiết các bước xử lý hình ảnh, tăng cường dữ liệu, chuẩn hóa văn bản, xây dựng OCR cache và trạng thái khảo sát ablation.
- **Mục 5 (Phương pháp thực hiện):** Thiết kế chi tiết và cấu hình triển khai của cả ba pipeline.
- **Mục 6 (Thực nghiệm):** Trình bày cấu hình, phạm vi dữ liệu, protocol đánh giá và giới hạn bằng chứng huấn luyện.
- **Mục 7 (Kết quả thực nghiệm):** Thảo luận kết quả all-samples, present-only, latency thành phần và phân tích lỗi tái lập.
- **Mục 8 (Kết luận):** Trả lời các câu hỏi nghiên cứu, đúc rút hạn chế và mở ra hướng phát triển tương lai.

2. Cơ sở lý thuyết

Chương này trình bày các cơ sở lý thuyết nền tảng cho đồ án, bao gồm bài toán trích xuất thông tin tài liệu (DIE), tổng quan về các công trình nghiên cứu liên quan, cơ chế hoạt động của nhận dạng ký tự quang học (OCR), và chi tiết hai kiến trúc mô hình học sâu đại diện: DONUT (OCR-free) và LAYOUTXLM (OCR-based). Cuối cùng, chương định nghĩa các độ đo đánh giá cùng quy ước xử lý đặc biệt đối với giá trị rỗng.

2.1. Bài toán trích xuất thông tin tài liệu

Trích xuất thông tin tài liệu (Document Information Extraction - DIE) hay trích xuất thông tin then chốt (Key Information Extraction - KIE) là bài toán nhận dạng và trích xuất các thực thể thông tin có cấu trúc dưới dạng cặp khóa-giá trị (key-value pairs) từ hình ảnh tài liệu bán cấu trúc (như biên lai, hóa đơn, tờ khai).

Đầu vào là một ảnh biên lai $I \in \mathbb{R}^{H \times W \times C}$. Đầu ra mong muốn là một tập hợp các thực thể thông tin: $Y = \{(K_1, V_1), (K_2, V_2), \dots, (K_M, V_M)\}$, trong đó mỗi K_j tương ứng với một trường thông tin đích (ví dụ: **store_name**, **date**, **total**, **address**) và V_j là văn bản đã chuẩn hóa được trích xuất từ tài liệu. Quá trình này yêu cầu mô hình đồng thời nhận diện được ký tự văn bản và hiểu được mối quan hệ không gian, ngữ nghĩa giữa các vùng chữ để liên kết chúng một cách chính xác.

2.2. Các công trình nghiên cứu liên quan

Bài toán trích xuất thông tin tài liệu đã đạt được nhiều bước tiến lớn nhờ sự phát triển của các kiến trúc Transformer và mô hình đa phương thức. [Bảng 2.1](#) so sánh tổng quan ba hướng tiếp cận được khảo sát trong đồ án cùng giả thuyết thực nghiệm đi kèm.

Các nghiên cứu nền tảng và công trình liên quan trực tiếp đến đồ án bao gồm:

- **MC-OCR 2021:** Cuộc thi xử lý biên lai chụp di động do Zalo AI tổ chức [12], cung cấp bộ dữ liệu biên lai tiếng Việt thực tế làm cơ sở cho đồ án. Các giải pháp hàng đầu trong cuộc thi thường kết hợp bộ phát hiện chữ (DBNet, YOLO) và bộ nhận dạng mạnh mẽ, kết hợp với các luật hậu xử lý sâu.
- **CORD:** Bộ dữ liệu biên lai quốc tế Consolidated Receipt Dataset [9], đóng vai trò benchmark chính cho các kiến trúc hiểu tài liệu end-to-end.
- **LayoutLM và LayoutLMv3:** Các mô hình tiền huấn luyện đa phương thức của Microsoft [3], [10]. LAYOUTXLM [11] là phiên bản mở rộng đa ngôn ngữ dựa trên XLM-RoBERTa, đóng vai trò cốt lõi trong pipeline OCR-based tiếng Việt của đồ án.
- **TrOCR:** Mô hình nhận dạng chữ viết (Text Recognition) dựa hoàn toàn trên kiến trúc Transformer Encoder-Decoder [6]. TrOCR nhận đầu vào là một ảnh dòng chữ được cắt ra từ bộ phát hiện chữ viết độc lập (Text Detector) và giải mã thành văn bản, đạt hiệu năng rất cao nhờ cơ chế tự chú ý.

Bảng 2.1: So sánh đặc tính và giả thuyết kiểm chứng của ba hướng tiếp cận.

Phương pháp	Đặc trưng kỹ thuật	Ưu điểm & Hạn chế	Giả thuyết kiểm chứng
Baseline (Regex)	Đối sánh từ khóa và biểu thức chính quy (Regex) trên kết quả OCR.	Tốc độ xử lý cực nhanh, không cần dữ liệu huấn luyện. Tuy nhiên, luật heuristics cứng nhắc, dễ thất bại trước bố cục đa dạng.	Hoạt động hiệu quả trên các trường có định dạng cấu trúc cố định (date , total); kém hiệu quả trên các trường tự do (store_name , address).
LAYOUTXMLM (OCR-based)	Phân loại token (BIO tagging) đa phương thức: Text + Layout (2D BBox) + Visual.	Kết hợp văn bản, vị trí và ảnh trong cùng encoder. Tuy nhiên, pipeline phức tạp, phụ thuộc trực tiếp vào chất lượng và thứ tự đọc của OCR đầu vào.	Có khả năng khai thác bố cục 2D, nhưng dễ bị đứt gãy thực thể đa dòng và phụ thuộc vào độ phủ của nhãn hộp bao.
DONUT (OCR-free)	Mô hình sinh (generative) end-to-end từ ảnh sang văn bản có cấu trúc (thẻ XML).	Loại bỏ bộ OCR độc lập và tránh truyền lỗi từ OCR rời rạc. Tuy nhiên, giải mã tự hồi quy có thể sinh chuỗi sai cấu trúc, lặp hoặc không có căn cứ trong ảnh.	Cần kiểm tra mức độ ổn định của chuỗi sinh và khả năng học từ tập dữ liệu tiếng Việt quy mô hạn chế.

2.3. Nhận dạng ký tự quang học (OCR)

Hệ thống OCR hiện đại thường được thiết kế thành một pipeline gồm hai giai đoạn độc lập: phát hiện văn bản và nhận dạng chữ viết. Trong pipeline OCR-based của đồ án, module OCR đảm nhận vai trò trích xuất văn bản thô cùng tọa độ 2D từ ảnh biên lai để làm đầu vào cho mô hình LAYOUTXLM.

2.3.1. Phát hiện văn bản với PaddleOCR

Pipeline sử dụng module phát hiện văn bản của PADDLEOCR [2]; phần nhận dạng tích hợp của PaddleOCR bị tắt. Họ PP-OCR sử dụng bộ phát hiện dựa trên Differentiable Binarization (DB), trong đó bước nhị phân hóa được đưa vào mạng dưới dạng khả vi:

$$\hat{B}_{i,j} = \frac{1}{1 + \exp(-k(P_{i,j} - T_{i,j}))} \quad (2.1)$$

trong đó $P_{i,j}$ là bản bản đồ xác suất chứa ký tự, $T_{i,j}$ là bản đồ ngưỡng thích ứng học được từ mạng backbone và FPN, và k là hệ số phóng đại khả vi. Cơ chế này cho phép tối ưu hóa trực tiếp ranh giới hộp bao văn bản thông qua lan truyền ngược, đem lại độ chính xác cao đối với các dòng chữ nhỏ và nghiêng trên biên lai.

2.3.2. Nhận dạng văn bản tiếng Việt với VietOCR

Ảnh của các dòng chữ sau khi được phát hiện được đưa vào VIETOCR [8]. Cấu hình vgg_transformer sử dụng mạng CNN họ VGG để trích xuất đặc trưng thị giác, sau đó biến đổi bản đồ đặc trưng thành một chuỗi để Transformer encoder-decoder sinh văn bản Unicode tiếng Việt theo cơ chế tự hồi quy.

Tại bước giải mã thứ t , mô hình dự đoán ký tự kế tiếp dựa trên đặc trưng ảnh $\mathbf{x}$ và các ký tự đích trước đó. Hàm mất mát là cross-entropy theo chuỗi, có label smoothing trong triển khai huấn luyện:

$$\mathcal{L}_{\text{VietOCR}} = - \sum_{t=1}^T \log P(y_t | y_{<t}, \mathbf{x}). \quad (2.2)$$

Vì vậy VietOCR trong đồ án không sử dụng hàm mất mát CTC. Sự phân biệt này quan trọng khi phân tích lỗi: kết quả nhận dạng phụ thuộc vào quá trình giải mã tuần tự chứ không chỉ vào căn chỉnh đơn điệu giữa đặc trưng ảnh và ký tự.

2.4. Mô hình Donut (OCR-free)

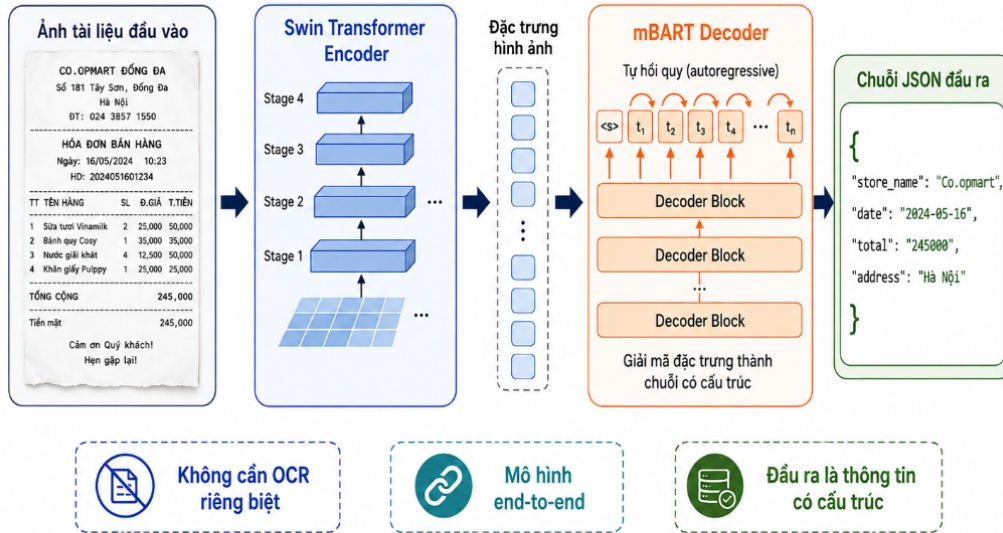
DONUT (Document Understanding Transformer) [4] là kiến trúc Transformer encoder-decoder ánh xạ trực tiếp điểm ảnh của tài liệu thành chuỗi có cấu trúc mà không cần module OCR trung gian. Trong đồ án, cấu trúc đích dùng các token mở/đóng tương tự XML.

2.4.1. Kiến trúc tổng quan

Kiến trúc Donut bao gồm hai thành phần chính:

1. **Vision Encoder (Swin Transformer):** Mã hóa hình ảnh biên lai thành chuỗi biểu diễn đặc trưng thị giác phân cấp $\mathbf{z} = \{z_1, z_2, \dots, z_n\}$. Swin Transformer [7] sử dụng cơ chế cửa sổ tự chú ý trượt (Shifted Window MSA) để giảm độ phức tạp tính toán từ bậc hai xuống bậc tuyến tính theo kích thước ảnh, đồng thời duy trì sự liên tục ngữ cảnh.
2. **Text Decoder (mBART):** Checkpoint donut-base dùng decoder thuộc họ mBART, một biến thể đa ngôn ngữ của kiến trúc BART [5]. Tại mỗi bước thời gian i , decoder kết hợp tự chú ý trên các token đã sinh và cross-attention lên biểu diễn thị giác $\mathbf{z}$ để dự đoán token tiếp theo:

$$\mathcal{L}_{\text{Donut}} = - \sum_{i=1}^T \log P(y_i | y_{<i}^*, \mathbf{z}) \quad (2.3)$$



Hình 2.1: Kiến trúc tổng quan mô hình DONUT (Swin Encoder kết hợp mBART Decoder) [4].

2.4.2. Phân biệt khái niệm Warm-up trong Donut

Trong quá trình huấn luyện mô hình Donut, cần phân biệt rõ hai khái niệm "warm-up" để gây nhầm lẫn:

- **Learning Rate Warm-up (tỷ lệ $warmup_ratio$ hoặc số bước $warmup_steps$):** Đây là một kỹ thuật điều tiết tốc độ học trong tối ưu hóa (LR scheduling). Trong số bước đầu tiên (thường

là 10% tổng số bước huấn luyện), tốc độ học được tăng dần tuyến tính từ 0 lên mức cực đại được thiết lập nhằm tránh hiện tượng bùng nổ gradient khi bắt đầu cập nhật trọng số.

- **Pre-training/Warm-up mô hình trên tập CORD:** Đây là một nhánh pre-finetuning tùy chọn, trong đó mô hình có thể được huấn luyện thêm trên dữ liệu biên lai CORD trước khi fine-tune tiếng Việt. Nhánh này có trong mã nguồn nhưng không được sử dụng để tạo các kết quả chính của báo cáo.

2.5. Mô hình LayoutXLM (OCR-based)

LAYOUTXLM [11] là mô hình đa phương thức đa ngôn ngữ cho tài liệu có bố cục phong phú. Mô hình mở rộng kiến trúc đa phương thức của LayoutLMv2 bằng backbone văn bản XLM-RoBERTa [1], nhờ đó tiếp nhận văn bản đa ngôn ngữ cùng thông tin bố cục và ảnh.

2.5.1. Nhúng đa phương thức (Multimodal Embedding)

LayoutXLM tiếp nhận đồng thời chuỗi token OCR, tọa độ 2D và ảnh toàn trang. Ba nguồn biểu diễn chính gồm:

1. **Nhúng văn bản (Text Embedding):** Mã hóa đặc trưng ngữ nghĩa của từ.
2. **Nhúng không gian 2D (Spatial Embedding):** Tọa độ bounding box $[x_1, y_1, x_2, y_2]$ của từ được chuẩn hóa về khoảng $[0, 1000]$ và nhúng thông qua các bảng nhúng tọa độ để mô hình nắm bắt vị trí tương đối và tuyệt đối trên ảnh:

$$\mathbf{e}_{2D}(\text{bbox}_i) = \mathbf{e}_{x_1}(x_1) + \mathbf{e}_{y_1}(y_1) + \mathbf{e}_{x_2}(x_2) + \mathbf{e}_{y_2}(y_2) + \mathbf{e}_w(w) + \mathbf{e}_h(h) \quad (2.4)$$

3. **Nhúng thị giác (Visual Embedding):** Ảnh toàn trang được đưa qua backbone ResNeXt-FPN để tạo bản đồ đặc trưng, sau đó được pooling thành một chuỗi visual token. Các visual token mang nhúng bố cục riêng và được đưa vào Transformer cùng chuỗi token văn bản; hệ thống không cắt riêng từng hộp từ để tạo một vector ảnh cho từng token.

Đối với token văn bản, embedding đầu vào là tổng của nhúng từ, nhúng vị trí 1D và nhúng bố cục 2D. Đối với visual token, embedding thị giác được kết hợp với nhúng bố cục tương ứng. Hai chuỗi được nối và xử lý chung trong Transformer, cho phép trao đổi thông tin giữa văn bản và hình ảnh.

2.5.2. Spatial-aware Self-Attention

Cơ chế tự chú ý nhận biết không gian, được kế thừa từ LayoutLMv2, bổ sung các bias vị trí 1D và 2D dựa trên quan hệ tương đối giữa các bounding box của token i và token j :

$$\alpha_{ij} = \frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d}} + b^{1D}(j - i) + b_x^{2D}(x_{1,i} - x_{1,j}) + b_y^{2D}(y_{1,i} - y_{1,j}) \quad (2.5)$$

Điều này giúp mô hình tự động học được mối liên hệ logic giữa các từ nằm gần nhau về mặt không gian địa lý trên ảnh tài liệu.

2.5.3. Gán nhãn BIO (BIO Tagging)

Để thực hiện trích xuất thông tin, bài toán được định nghĩa dưới dạng phân loại token (token classification). Mỗi token đầu vào được dự đoán một nhãn trong hệ thống nhãn **BIO** gồm 9 nhãn được mô tả chi tiết trong [Bảng 2.2](#).

Bảng 2.2: Hệ thống nhãn **BIO** cho 4 trường thông tin trích xuất.

Trường thông tin	Nhãn Bắt đầu (Begin)	Nhãn Bên trong (Inside)
Tên cửa hàng (store_name)	B-STORE_NAME	I-STORE_NAME
Ngày tháng (date)	B-DATE	I-DATE
Tổng tiền (total)	B-TOTAL	I-TOTAL
Địa chỉ (address)	B-ADDRESS	I-ADDRESS
Trường không quan tâm	O (Outside)	

2.6. Các độ đo đánh giá và quy ước xử lý rỗng

Để đánh giá chính xác chất lượng trích xuất, các dự đoán sau khi được chuẩn hóa sẽ được so sánh với nhãn ground-truth thông qua ba chỉ số chính: EM, NES, và CER.

2.6.1. Exact Match (EM)

EM đo tỷ lệ khớp chính xác hoàn toàn từng ký tự giữa dự đoán (*pred*) và nhãn thực (*gold*):

$$EM(pred, gold) = \begin{cases} 1.0 & \text{nếu } pred.strip() = gold.strip() \\ 0.0 & \text{ngược lại} \end{cases} \quad (2.6)$$

Quy ước giá trị rỗng: Nếu cả *pred* và *gold* đều là chuỗi rỗng sau chuẩn hóa $\rightarrow EM = 1.0$. Nếu chỉ một trong hai chuỗi rỗng $\rightarrow EM = 0.0$.

2.6.2. Normalized Edit Similarity (NES)

NES đo mức độ tương đồng chuỗi ký tự thông qua khoảng cách Levenshtein chuẩn hóa; chỉ số này không đo tương đồng ngữ nghĩa:

$$NES(pred, gold) = 1 - \frac{\text{Levenshtein}(pred, gold)}{\max(\text{len}(pred), \text{len}(gold))} \quad (2.7)$$

Quy ước giá trị rỗng: Nếu cả *pred* và *gold* đều là chuỗi rỗng $\rightarrow NES = 1.0$ (tránh lỗi chia cho 0). Nếu chỉ một trong hai chuỗi rỗng $\rightarrow NES = 0.0$.

2.6.3. Character Error Rate (CER)

CER đo tỷ lệ lỗi ở cấp độ ký tự trên chuỗi nhãn thực:

$$CER(pred, gold) = \frac{\text{Levenshtein}(pred, gold)}{\text{len}(gold)} \quad (2.8)$$

Quy ước giá trị rỗng:

- Nếu cả hai cùng rỗng $\rightarrow CER = 0.0$.
- Nếu *gold* rỗng nhưng *pred* có giá trị $\rightarrow CER = 1.0$.
- Nếu *gold* có giá trị nhưng *pred* rỗng $\rightarrow CER = 1.0$.

Giải thích hiện tượng CER > 100%: Chỉ số CER có thể vượt quá 100% (hay > 1.0) khi chuỗi dự đoán dài hơn đáng kể so với nhãn thực do mô hình sinh thừa nhiều ký tự rác hoặc bị lỗi lặp chuỗi tự hồi quy (hallucination), dẫn đến khoảng cách Levenshtein lớn hơn độ dài ký tự của nhãn thực *gold*.

2.6.4. Hai chế độ tổng hợp kết quả

Để tránh việc một mô hình nhận điểm do đồng thời trả về rỗng khi ground-truth cũng rỗng, báo cáo trình bày hai chế độ:

- **All samples:** đánh giá trên toàn bộ 234 mẫu, tuân thủ các quy ước giá trị rỗng ở trên.
- **Present-only:** với từng trường, chỉ đánh giá các mẫu có ground-truth khác rỗng. Đây là chế độ chính để phân tích năng lực trích xuất khi thông tin thực sự xuất hiện.

Macro EM, NES và CER đều là trung bình không trọng số của bốn trường.

3. Tập dữ liệu

Chương này mô tả dữ liệu processed thực tế được dùng trong báo cáo. Các thống kê được đọc từ ba tập `train.jsonl`, `val.jsonl` và `test.jsonl`; các số liệu về ảnh thô hoặc mẫu từng bị loại không được công bố khi không có artifact kiểm chứng đi kèm.

3.1. Nguồn dữ liệu

Dữ liệu gồm hai nguồn:

- **MC-OCR 2021** [12]: cung cấp ảnh, giá trị trường và hộp bao. Bốn nhãn nguồn được ánh xạ sang tên cửa hàng, địa chỉ, ngày và tổng tiền trong schema thống nhất.
- **Dữ liệu tự thu thập**: gồm ảnh biên lai tiếng Việt và nhãn JSON bốn trường. Trong processed dataset hiện tại, các mẫu này có `annotation_level=json_only`, không có hộp bao dùng được cho LayoutXLM.

Tổng dữ liệu processed gồm 1.559 mẫu: 1.120 mẫu MC-OCR và 439 mẫu tự thu thập.

3.2. Schema JSONL thống nhất

Bảng 3.1: Các trường chính trong schema JSONL.

Trường	Mô tả
ID và nhóm	<code>id</code> , <code>group_id</code> : định danh mẫu và nhóm ảnh liên quan.
Nguồn và cửa hàng	<code>source</code> , <code>store_group</code> : nguồn dữ liệu và nhóm cửa hàng.
Ảnh	<code>image_path</code> , <code>width</code> , <code>height</code> : đường dẫn và kích thước ảnh.
Mức annotation	<code>annotation_level</code> : <code>json_and_boxes</code> hoặc <code>json_only</code> .
Nhãn trường	<code>raw_target</code> , <code>target</code> : nhãn trước và sau chuẩn hóa.
Vùng trường	<code>field_boxes</code> , <code>field_polygons</code> : vùng không gian nếu có.
Oracle OCR	<code>oracle_ocr</code> : văn bản và hộp bao ground-truth khi có.
OCR cache	<code>ocr_cache_path</code> : đường dẫn tới cache PaddleOCR + VietOCR.

Các trường đích được chuẩn hóa như sau: **date** về YYYY-MM-DD, **total** thành chuỗi chữ số, còn **store_name** và **address** được chuẩn hóa Unicode NFC, khoảng trắng và tiền tố trường.

3.3. Chia dữ liệu và chống rò rỉ

Tập dữ liệu được chia theo tỷ lệ gần 70/15/15 với seed 42. Script chia dữ liệu nhóm theo **group_id**, phân tầng theo nguồn và có bước kiểm tra hash ảnh để giảm nguy cơ cùng một nội dung xuất hiện ở nhiều split.

Ba tập JSONL hiện tại là nguồn sự thật cho mọi thống kê train/validation/test trong báo cáo.

Bảng 3.2: Phân bố mẫu trong các tệp processed.

Split	MC-OCR	Tự thu thập	Tổng
Train	784	307	1.091
Validation	168	66	234
Test	168	66	234
Tổng	1.120	439	1.559

3.4. Mức annotation và dữ liệu huấn luyện

Bảng 3.3: Mức annotation theo split.

Split	json_and_boxes	json_only
Train	784	307
Validation	168	66
Test	168	66

Donut sử dụng toàn bộ train/validation. LayoutXLM lọc `json_and_boxes`, nên chỉ sử dụng 784 mẫu train và 168 mẫu validation. Cả ba phương pháp được chấm trên toàn bộ 234 mẫu test; các mẫu `json_only` vẫn có nhãn trường để đánh giá đầu ra JSON.

3.5. Giá trị ground-truth rỗng trên test

Bảng 3.4: Số mẫu có ground-truth rỗng trên 234 mẫu test.

Trường	Số mẫu rỗng	Tỷ lệ
store_name	10	4,27%
date	18	7,69%
total	17	7,26%
address	11	4,70%

Tỷ lệ này giải thích vì sao cần báo cáo present-only. Nếu mô hình luôn trả về rỗng, nó vẫn nhận EM all-samples đúng bằng tỷ lệ ground-truth rỗng của trường đó.

3.6. Kiểm tra hợp lệ

Các báo cáo validation hiện có ghi nhận 0 lỗi schema, 0 ngày sai định dạng và 0 tổng tiền sai định dạng trong cả ba split. Kết quả này chỉ xác nhận tính hợp lệ cú pháp của dữ liệu processed, không chứng minh nhãn đúng hoàn toàn về mặt nội dung.

4. Tiền xử lý dữ liệu

Chương này trình bày các bước xử lý hình ảnh và văn bản được cài đặt trong hệ thống. Các cấu hình được mô tả theo mã nguồn hiện tại; báo cáo không coi chúng là cấu hình tối ưu nếu chưa có thí nghiệm ablation đủ bằng chứng tái lập.

4.1. Tiền xử lý ảnh (Image Preprocessing)

Chất lượng ảnh biên lai đầu vào ảnh hưởng trực tiếp đến hiệu năng nhận dạng của bộ OCR. Pipeline hiện gọi profile mặc định `resize`; trong mã thực thi, profile này chỉ `resize` cạnh dài và không tự động gọi `denoise` hoặc `CLAHE`:

- **Resize hình ảnh:**

- Đối với pipeline OCR (Baseline và LayoutXLM): Ảnh được `resize` theo cạnh dài nhất với `max_long_side = 1600` pixel để bảo đảm nét chữ rõ ràng và duy trì tỷ lệ khung hình (aspect ratio).
- Đối với Donut: Ảnh được `resize` về kích thước cố định 1280×960 pixel, phần thiếu được lấp đầy bằng khoảng trắng (padding).

Tập cấu hình còn chứa các cờ `denoise=true` và `clahe=true`, nhưng pipeline tiền xử lý hiện chưa đọc hoặc thực thi hai cờ này. Vì vậy báo cáo coi chúng là cấu hình dự kiến, không phải bước đã áp dụng cho artifact. Mã nguồn thực thi hỗ trợ thêm:

- **Nhị phân hóa (Binarize):** `Resize` cạnh dài rồi áp dụng adaptive Gaussian threshold, không phải Otsu.
- **Nắn thẳng ảnh (Rectify):** Tìm contour bốn góc và dùng perspective transform; nếu không tìm được contour hợp lệ, ảnh gốc được giữ lại.

4.2. Tăng cường dữ liệu (Data Augmentation)

Tăng cường dữ liệu được áp dụng trực tuyến và chỉ trên tập train. Hai mô hình sử dụng hai nhóm phép biến đổi khác nhau vì LayoutXLM phải giữ tọa độ OCR và hộp nhãn đồng nhất với ảnh.

Bảng 4.1: Các phép tăng cường dữ liệu được cài đặt cho hai mô hình.

Mô hình	Biến đổi hình học	Biến đổi quang học
DONUT	Affine: dịch $\pm 2\%$, scale 0, 97–1, 03, xoay $\pm 8^\circ$; perspective 0, 01–0, 025. Không áp dụng phép biến đổi làm thay đổi hình học để giữ nguyên tính hợp lệ của bounding box.	Brightness/contrast, Gaussian blur, Gaussian noise và random shadow.
LAYOUTXLM		Brightness/contrast, Gaussian blur và random shadow.

4.3. Chuẩn hoá văn bản (Text Normalization)

Chuẩn hóa văn bản là bước bắt buộc để đồng nhất định dạng dữ liệu đầu ra. Mã nguồn không tự sửa chính tả, không bỏ dấu và không chuyển toàn bộ chuỗi sang chữ thường; vì vậy EM vẫn nhạy với khác biệt hoa/thường và dấu tiếng Việt.

1. Chuẩn hóa chung (General Normalization) Áp dụng cho toàn bộ các trường thông tin:

- **Unicode NFC:** Chuyển đổi toàn bộ văn bản về dạng chuẩn tổ hợp Unicode NFC để đồng nhất biểu diễn ký tự có dấu.
- **Whitespace:** Gộp các khoảng trắng liên tiếp và cắt khoảng trắng ở hai đầu chuỗi.
- **Ký tự đặc biệt:** Loại bỏ các ký tự điều khiển không in được.

2. Chuẩn hóa theo trường (Field-specific Normalization)

- **store_name:** Loại bỏ tiền tố cửa hàng hoặc siêu thị ở đầu chuỗi, có hỗ trợ biến thể không dấu.
- **address:** Loại bỏ tiền tố địa chỉ, địa chỉ hoặc address ở đầu chuỗi.
- **date:** Chuyển đổi nhiều định dạng ngày tự do về dạng chuẩn ISO YYYY-MM-DD. Nếu phân tích cú pháp thất bại, trả về chuỗi rỗng.
- **total:** Loại bỏ đơn vị tiền tệ ("đ", "đồng", "VND"), dấu chấm/phẩy phân cách hàng nghìn và các số 0 ở đầu.

Bảng 4.2 minh họa kết quả chuẩn hóa cho từng trường.

Bảng 4.2: Ví dụ chuẩn hoá văn bản theo trường thông tin.

Trường	Văn bản gốc trước chuẩn hóa	Văn bản sau chuẩn hóa
store_name	Cửa hàng CIRCLE K	CIRCLE K
address	Địa chỉ: 123 Nguyễn Trãi, Thanh Xuân, HN	123 Nguyễn Trãi, Thanh Xuân, HN
date	Lúc 15/03/2021 14:30:15	2021-03-15
total	Tổng cộng: 115.000 đ	115000

4.4. Xây dựng OCR Cache

Quá trình chạy mô hình OCR (PaddleOCR kết hợp VietOCR) tốn nhiều tài nguyên tính toán và thời gian. Việc xây dựng OCR cache giúp lưu trữ lại kết quả nhận diện của từng ảnh để sử dụng lại tức thì trong các lượt huấn luyện và thực nghiệm khác nhau, đẩy nhanh đáng kể tốc độ thử nghiệm.

OCR cache lưu danh sách từ, bounding box, thông tin kích thước ảnh sau tiền xử lý và metadata phiên bản OCR. Cache được dùng chung cho Baseline và LayoutXLM nhằm tránh

chạy lại OCR trong huấn luyện, suy luận kiểm thử và phân tích lỗi. Báo cáo không công bố thời gian OCR CPU/GPU vì artifact hiện tại không chứa log đo đầy đủ để xác minh các con số đó.

4.5. Trạng thái khảo sát tiền xử lý OCR

Script khảo sát hỗ trợ bốn profile `none`, `resize`, `binarize` và `rectify`, dự kiến chạy trên `validation` thay vì `test` để tránh lựa chọn cấu hình dựa trên tập kiểm thử. Tuy nhiên, các CSV được lưu trong repository hiện chỉ chứa ba mẫu cho mỗi profile và không tái lập được bảng 100 mẫu từng được ghi trong bản thảo. Vì vậy, báo cáo loại bỏ bảng xếp hạng ablation và không khẳng định profile `resize` là tối ưu.

Profile `resize` được giữ làm cấu hình mặc định vì đây là lựa chọn kỹ thuật đã được tích hợp nhất quán trong OCR cache và inference. Một thí nghiệm ablation hợp lệ trong tương lai cần cố định danh sách mẫu `validation`, bảo đảm đủ cache cho từng profile, báo cáo cả độ phủ cache và không cộng điểm tự động cho trường `ground-truth` rỗng.

5. Phương pháp thực hiện

Chương này trình bày chi tiết thiết kế hệ thống và phương pháp thực hiện của ba hướng tiếp cận (pipeline) khác nhau nhằm giải quyết bài toán trích xuất thông tin biên lai tiếng Việt, bao gồm phương pháp Baseline dựa trên biểu thức chính quy, mô hình sinh End-to-End Donut, và mô hình phân loại chuỗi LayoutXLM kết hợp nhận dạng văn bản VietOCR.

5.1. Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế dạng module hóa cho phép kiểm thử và so sánh độc lập các kiến trúc. Sơ đồ kiến trúc tổng quan của ba phương pháp được mô tả chi tiết trong [Hình 5.1](#). Hệ thống nhận đầu vào là ảnh biên lai và đầu ra là tệp JSON chứa bốn trường thông tin: **store_name**, **date**, **total**, **address**.

- **Pipeline 1 — Baseline (Rule-based):** Ảnh → OCR (PaddleOCR + VietOCR) → Luật / Regex → JSON.
- **Pipeline 2 — DONUT (OCR-free):** Ảnh → Swin Encoder → mBART Decoder → JSON.
- **Pipeline 3 — VietOCR + LAYOUTXLM (OCR-based):** Ảnh → OCR → Token + BBox → LAYOUTXLM → BIO → JSON.

5.2. Baseline: OCR + Trích xuất dựa trên luật (Rule-based)

Phương pháp Baseline hoạt động như một hệ chuyên gia đơn giản. Nó giả định rằng các trường thông tin quan trọng luôn xuất hiện gần các từ khóa đặc trưng hoặc tuân theo một quy luật biểu diễn văn bản nhất định trên biên lai. Phương pháp này sử dụng các quy tắc thủ công (heuristic) và biểu thức chính quy (regular expression — regex) để trích xuất thông tin từ kết quả OCR. Đây là phương pháp đơn giản nhất, không yêu cầu huấn luyện mô hình học sâu, nhưng phụ thuộc mạnh vào chất lượng OCR và tính nhất quán của bố cục biên lai.

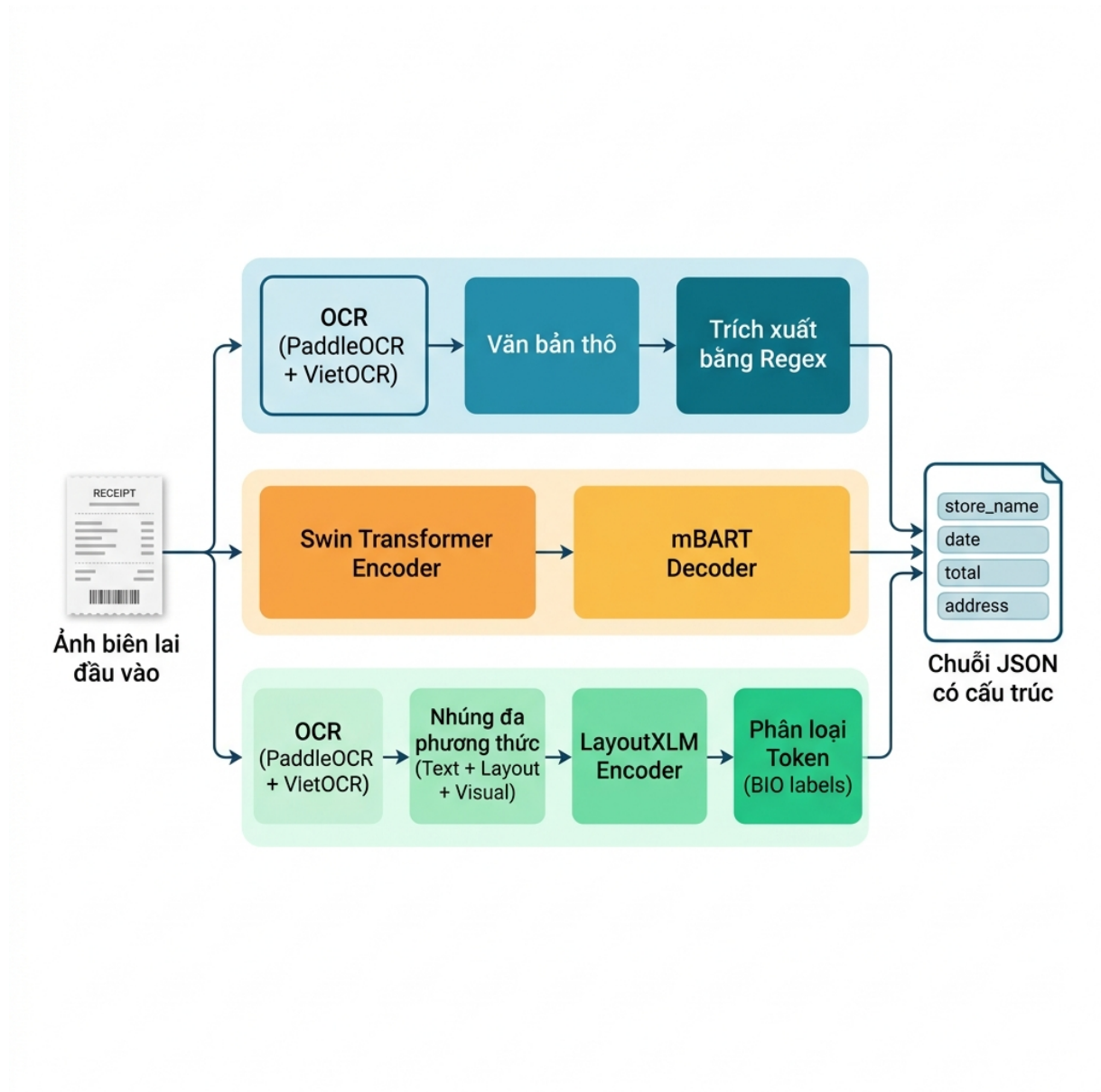
5.2.1. Pipeline tổng quan

Quy trình xử lý gồm hai bước chính:

1. **Nhận dạng văn bản:** Ảnh biên lai được đưa qua bộ phát hiện vùng chữ PaddleOCR [2] và bộ nhận dạng VietOCR [8] để thu được danh sách các cặp (văn bản, bounding box) sắp xếp theo thứ tự đọc (reading order).
2. **Trích xuất theo luật:** Áp dụng các quy tắc heuristic và regex riêng cho từng trường thông tin để trích xuất giá trị từ danh sách văn bản.

5.2.2. Quy tắc trích xuất cụ thể

Logic hoạt động cụ thể cho từng trường như sau:



Hình 5.1: Sơ đồ tổng quan kiến trúc ba pipeline trích xuất thông tin biên lai.

- **Tên cửa hàng (store_name):** Chấm điểm tối đa 5 dòng đầu theo vị trí, tỷ lệ chữ hoa và độ dài. Các dòng chứa từ khóa hóa đơn, mã số thuế, điện thoại hoặc địa chỉ, dòng quá ngắn và dòng có tỷ lệ chữ số cao bị loại.
- **Ngày tháng (date):** Duyệt từ trên xuống, chuẩn hóa từng dòng và lấy ngày hợp lệ đầu tiên trong khoảng năm 2010–2030. Bộ chuẩn hóa hỗ trợ dấu /, -, . và năm hai chữ số.
- **Tổng tiền (total):** Duyệt từ cuối hóa đơn lên để ưu tiên từ khóa tổng tiền gần phần thanh toán. Với mỗi dòng có từ khóa, xét dòng đó và tối đa một dòng kế tiếp, lọc số điện thoại, mã số thuế, ngày và giá trị dưới 1.000 rồi chọn số tiền lớn nhất. Nếu không có từ khóa, thuật toán tìm ứng viên hợp lệ trong một phần ba cuối hóa đơn.
- **Địa chỉ (address):** Tìm dòng có từ khóa địa chỉ nhưng không chứa ngày, điện thoại, mã số thuế hoặc từ khóa thanh toán. Nếu dòng kế tiếp không bị chặn và có nội dung đủ dài, nối thêm tối đa dòng này trước khi chuẩn hóa.

Các quy tắc trích xuất được thiết kế dựa trên quan sát thực nghiệm trên tập dữ liệu biên lai tiếng Việt. [Bảng 5.1](#) tóm tắt các quy tắc chính cho từng trường.

Bảng 5.1: Quy tắc trích xuất cho từng trường thông tin trong pipeline baseline.

Trường	Mô tả quy tắc
store_name	Chấm điểm tối đa 5 dòng đầu; loại các dòng giống tiêu đề hóa đơn, mã số thuế, điện thoại, địa chỉ hoặc chứa quá nhiều chữ số.
date	Tìm chuỗi khớp regex ngày tháng, bỏ qua các từ khóa: ngày, date, time.
total	Duyệt từ cuối lên, xét dòng có từ khóa và một dòng kế tiếp; nếu không có từ khóa thì tìm số tiền hợp lệ trong một phần ba cuối.
address	Tìm dòng có từ khóa địa chỉ, loại ngữ cảnh bị chặn và nối tối đa một dòng kế tiếp hợp lệ.

Các mẫu regex được sử dụng cho trường **date** và **total** được trình bày trong [Đoạn mã 1](#).

```

1 # --- Regex cho trường date ---
2 DATE_PATTERNS = [
3     r'(\d{1,2})[/-](\d{1,2})[/-](\d{4})',      # dd/mm/yyyy hoặc dd-mm-
4     r'(\d{1,2})[/-](\d{1,2})[/-](\d{2})',      # dd/mm/yy
5     r'(\d{4})[/-](\d{1,2})[/-](\d{1,2})',      # yyyy/mm/dd (ISO)
6 ]
7 DATE_IGNORE_KW = ['ngày', 'date', 'time']
8
9 # --- Regex cho trường total ---
10 TOTAL_KEYWORDS = ['tong cong', 'thanh toan', 'total', 'phai tra']
11 MONEY_PATTERN = r'\d[\d\.\s]*'                # so tien: 150.000, 1,500,000
12 TOTAL_SEARCH_RANGE = 1                        # dòng keyword và một dòng kế
    tiếp

```

Đoạn mã 1: Các mẫu regex dùng trong pipeline baseline.

5.3. Donut: End-to-End OCR-free

Mô hình Donut được lựa chọn nhằm kiểm thử giả thuyết liệu một mạng nơ-ron có thể tự động học cách ”đọc” và ”hiểu” bố cục biên lai trực tiếp từ điểm ảnh mà không cần thông qua bước nhận dạng ký tự quang học rời rạc hay không. Tiếp cận này giúp loại bỏ hoàn toàn sự phụ thuộc vào chất lượng của bộ máy OCR độc lập. DONUT (Document Understanding Transformer) [4] là mô hình end-to-end không cần bước OCR riêng biệt. Mô hình sử dụng kiến trúc encoder-decoder: encoder là Swin Transformer [7] trích xuất đặc trưng hình ảnh, decoder là [Bidirectional and Auto-Regressive Transformers \(BART\)](#) [5] sinh chuỗi văn bản đầu ra chứa thông tin cần trích xuất.

5.3.1. Chuẩn bị dữ liệu

Định dạng chuỗi đích sử dụng cấu trúc thẻ tương tự XML giúp biểu diễn mối quan hệ phân cấp và nhãn của từng trường thông tin một cách tường minh. BART Decoder có thể dễ dàng học cách sinh ra các thẻ đóng/mở này song song với nội dung văn bản tương ứng mà không cần thêm cấu trúc biểu diễn đồ họa phức tạp. Mỗi mẫu huấn luyện là một cặp (ảnh biên lai, chuỗi đích). Chuỗi đích được định dạng bằng các token đặc biệt đánh dấu từng trường thông tin. Ảnh đầu vào được resize về kích thước 1280×960 pixel, giữ nguyên tỷ lệ khung hình (aspect ratio) bằng cách thêm padding.

Ví dụ chuỗi đích cho một biên lai:

```
1 <s_receipt_ie>
2   <s_store_name>CUA HANG TIEN LOI GS25</s_store_name>
3   <s_date>15/03/2024</s_date>
4   <s_total>125.000</s_total>
5   <s_address>123 Nguyen Trai, Thanh Xuan, Ha Noi</s_address>
6 </s_receipt_ie>
```

Đoạn mã 2: Ví dụ chuỗi đích (target string) cho mô hình DONUT.

5.3.2. Cấu hình huấn luyện

Các siêu tham số của Donut được cấu hình cho GPU Tesla T4 16GB. Kích thước batch thô là 2, kết hợp tích lũy gradient 8 bước để đạt batch hiệu dụng 16. Đây là cấu hình chạy được lưu trong repository, không phải kết quả của một quá trình tối ưu siêu tham số có kiểm soát. [Bảng 5.2](#) trình bày cấu hình huấn luyện của mô hình DONUT.

Bảng 5.2: Cấu hình huấn luyện mô hình DONUT.

Tham số	Giá trị
Pretrained model	naver-clova-ix/donut-base
Max length	768 tokens
Epochs	150
Batch size	2
Gradient accumulation	8 (effective batch size = 16)
Learning rate	2×10^{-5}
LR scheduler	Cosine
Warmup ratio	0.1
FP16	Có
Gradient checkpointing	Có
Early stopping patience	15 epochs

5.3.3. Suy luận (Inference)

Trong quá trình suy luận, Donut nhận đầu vào là ảnh biên lai và prompt `<s_receipt_ie>`. Decoder sinh tuần tự các token tiếp theo. Bộ phân tích cú pháp tìm nội dung giữa các token mở/đóng; trường không tạo được cặp token hợp lệ được gán chuỗi rỗng trước khi chuẩn hóa. Trong giai đoạn suy luận, mô hình DONUT nhận ảnh biên lai làm đầu vào và sinh chuỗi văn bản thông qua beam search với `num_beams=2`. Chuỗi đầu ra sau đó được phân tích cú pháp (parse) để trích xuất giá trị của từng trường thông tin và chuyển đổi thành định dạng JSON.

Quy trình suy luận:

1. Ảnh biên lai được resize về 1280×960 và đưa qua encoder.
2. Decoder sinh chuỗi đầu ra sử dụng beam search (`num_beams=2`).
3. Chuỗi đầu ra được parse bằng regex để trích xuất nội dung giữa các cặp token `<s_field>` và `</s_field>`.
4. Kết quả được ghi vào tệp JSON.

Cấu hình suy luận Donut

Trong phiên bản thực nghiệm trước đây, do không truyền tham số `max_length` một cách tường minh khi gọi hàm sinh của thư viện Transformers, mô hình Donut đã kế thừa giá trị mặc định là 20 token, dẫn đến hầu hết chuỗi kết quả sinh ra bị cắt bớt (173/234 mẫu test chạm giới hạn này). Lỗi cấu hình này hiện đã được khắc phục hoàn toàn bằng việc truyền tường minh tham số `max_length=768` trong lệnh suy luận mới. Nhờ đó, artifact kết quả của Donut trong báo cáo hiện tại đại diện cho một lượt suy luận Donut đầy đủ và hợp lệ để đưa vào so sánh chất lượng và latency chính.

5.4. VietOCR + LayoutXLM: OCR-based Layout-aware

Tiếp cận kết hợp VietOCR và LayoutXLM tận dụng tối đa sức mạnh của mô hình đa phương thức. Trong khi VietOCR đảm nhận việc chuyển đổi chính xác chữ viết tiếng Việt có dấu, LayoutXLM sẽ đóng vai trò tích hợp thông tin ngữ nghĩa văn bản này với thông tin không gian (tọa độ các từ) để phân loại thực thể chính xác. Hướng tiếp cận thứ ba kết hợp kết quả OCR với mô hình LAYOUTXLM [11] — một mô hình đa phương thức (multimodal) được tiền huấn luyện trên dữ liệu đa ngôn ngữ, có khả năng khai thác đồng thời ba loại thông tin: văn bản (text), bố cục không gian (layout/bounding box) và hình ảnh (visual features). Bài toán trích xuất thông tin được quy về bài toán gán nhãn chuỗi (sequence labeling) sử dụng lược đồ BIO.

5.4.1. Chuẩn bị dữ liệu

LayoutXLM chỉ sử dụng các mẫu có `annotation_level=json_and_boxes`. Với mỗi từ OCR, mã nguồn tính tỷ lệ phần diện tích của hộp từ nằm trong hộp ground-truth:

$$\text{coverage}(W, F) = \frac{|W \cap F|}{|W|}. \quad (5.1)$$

Đây là tỷ lệ bao phủ hộp từ, không phải Intersection over Union. Từ được gán vào trường có coverage lớn nhất nếu tỷ lệ đạt ít nhất 0,5; các từ còn lại nhận nhãn $\emptyset$. **Đầu vào:** Kết quả OCR từ cache bao gồm văn bản và bounding box cho từng từ (word-level).

Gán nhãn BIO: Các từ liên tiếp cùng thuộc một trường nhận lần lượt nhãn B-FIELD và I-FIELD. Khi một từ không đạt ngưỡng coverage, chuỗi thực thể hiện tại kết thúc.

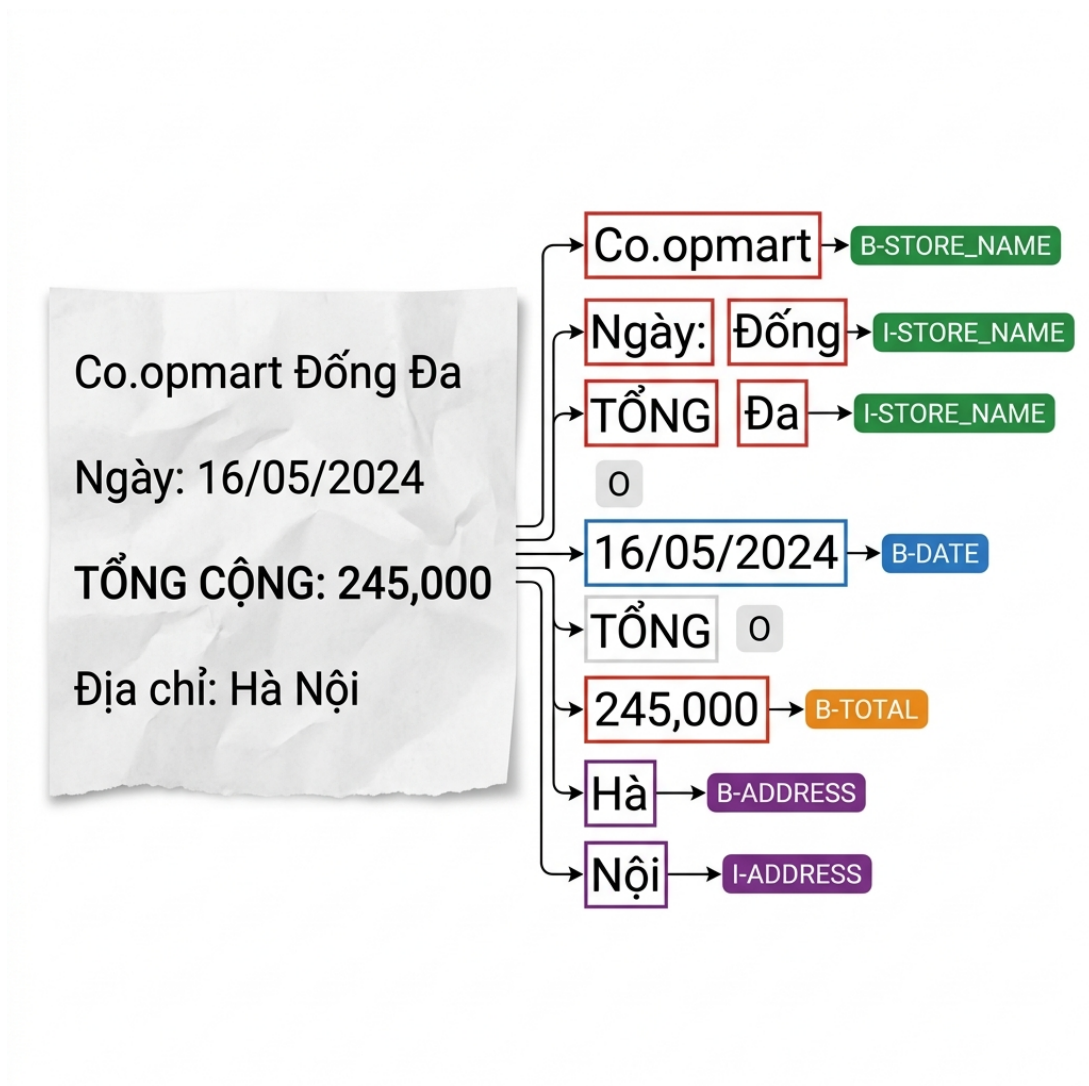
Tokenization: Văn bản được tokenize với giới hạn 512 token. Tham số `doc_stride` được đặt là 128 trong cấu hình, nhưng dataset và inference hiện chưa tạo sliding windows. Chuỗi dài bị cắt ở giới hạn token, vì vậy các trường nằm cuối biên lai có thể bị mất.

5.4.2. Cấu hình huấn luyện

Siêu tham số của LayoutXLM được lấy từ cấu hình thực nghiệm trong repository. Số epoch tối đa là 20 và early stopping patience là 5; báo cáo không khẳng định mô hình đã chạy đủ hoặc dừng tại một epoch cụ thể vì log huấn luyện đầy đủ không còn trong artifact hiện tại. [Bảng 5.3](#) trình bày cấu hình huấn luyện của mô hình LAYOUTXLM.

Hệ thống nhãn BIO gồm 9 nhãn được mô tả chi tiết trong [Bảng 2.2](#) ở [Mục 2](#).

Tập train LayoutXLM gồm 784 mẫu và validation gồm 168 mẫu có hộp bao. Các mẫu `json_only` không thể tham gia huấn luyện token classification theo quy trình này. Donut không cần hộp bao nên sử dụng toàn bộ 1.091 mẫu train và 234 mẫu validation.



Hình 5.2: Ví dụ gán nhãn BIO cho các từ trên biên lai.

Bảng 5.3: Cấu hình huấn luyện mô hình LAYOUTXLM.

Tham số	Giá trị
Pretrained model	microsoft/layoutxlm-base
Max length	512 tokens
Doc stride	128 (đã khai báo, chưa dùng sliding window)
Số nhãn (labels)	9 nhãn BIO
Epochs	20
Batch size	8
Gradient accumulation	4 (effective batch size = 32)
Learning rate	2×10^{-5}
LR scheduler	Linear
Warmup ratio	0.1
FP16	Có
Early stopping patience	5 epochs

5.4.3. Suy luận và hậu xử lý (Inference & Post-processing)

Thuật toán gộp nhãn hoạt động như một máy trạng thái: B-FIELD mở span mới; I-FIELD cùng loại nối vào span đang mở. Nếu gặp I-FIELD khi không có span cùng loại, triển khai hiện tại vẫn mở một span mới để không làm mất thực thể, tương đương chính sách sửa chuỗi BIO bị đứt. Nhãn O hoặc nhãn của trường khác đóng span hiện tại. Quy trình suy luận của pipeline LAYOUTXLM:

1. **Dự đoán nhãn BIO:** Mô hình LAYOUTXLM nhận đầu vào gồm token, bounding box và ảnh, sau đó dự đoán nhãn BIO cho từng token.
2. **Gộp span và chọn ứng viên:** Sau khi tạo span, tên cửa hàng lấy span đầu tiên; ngày lấy span đầu tiên chuẩn hóa hợp lệ; tổng tiền loại số giống điện thoại/MST hoặc quá nhỏ rồi chọn giá trị lớn nhất; các span địa chỉ gần nhau theo trục y được ghép thành block và ưu tiên block có từ khóa địa chỉ hoặc có độ dài lớn nhất.
3. **Chuẩn hoá:** Văn bản trích xuất được chuẩn hoá (loại bỏ khoảng trắng thừa, chuẩn hoá Unicode [Unicode Normalization Form Composed](#) — [Dạng chuẩn hoá Unicode tổ hợp \(NFC\)](#)) trước khi ghi vào tệp JSON đầu ra.

5.5. Hậu xử lý chung (Post-processing)

Hậu xử lý chung chuẩn hóa schema và một số định dạng trường trước khi đánh giá. Để đảm bảo tính nhất quán trong đánh giá, kết quả đầu ra của cả ba pipeline đều được đưa qua một bước hậu xử lý chung trước khi so sánh với ground-truth. Các bước hậu xử lý bao gồm:

- **Chuẩn hoá Unicode:** Chuyển đổi toàn bộ văn bản về dạng NFC để thống nhất cách biểu diễn ký tự tiếng Việt.
- **Chuẩn hoá khoảng trắng:** Loại bỏ khoảng trắng đầu/cuối và thay thế các chuỗi khoảng trắng liên tiếp bằng một khoảng trắng đơn.
- **Định dạng thống nhất:** Đảm bảo tất cả các trường đầu ra đều có mặt trong tệp JSON, sử dụng chuỗi rỗng cho các trường không trích xuất được.

Triển khai hiện tại không chuyển **store_name** và **address** sang chữ thường; EM vì vậy phân biệt hoa/thường. **date** được parse về ISO và **total** chỉ giữ chữ số.

6. Thiết lập thực nghiệm

Chương này mô tả cấu hình được lưu trong repository, phạm vi dữ liệu của từng mô hình và quy trình tạo các artifact đánh giá. Những thông tin không còn log gốc để xác minh, như thời gian huấn luyện hoặc epoch dừng thực tế, không được trình bày như kết quả đo.

6.1. Trạng thái prediction artifact

Bảng 6.1: Trạng thái bằng chứng của các prediction artifact đã khóa bằng SHA-256.

Artifact	Trạng thái	n	SHA-256	Phạm vi sử dụng
Baseline	Hợp lệ	234	a16c4d903e...	Bảng chính, RQ1, latency thành phần
VietOCR + LAYOUTXLM	Hợp lệ, kèm sensitivity check	234	c65ffdca7f...	Bảng chính, RQ1, latency thành phần
DONUT	Không hợp lệ; chỉ chẩn đoán	234	1550902cb1...	Phụ lục/chẩn đoán; không xếp hạng

Cả ba phương pháp Baseline, LayoutXLM và Donut đều có đủ 234 bản ghi dự đoán trên tập test, không có bản ghi nào bị lỗi (`status=error`) và đều được đưa vào so sánh chính. Với LayoutXLM, báo cáo bổ sung sensitivity check loại bốn ID có sai lệch tọa độ ảnh. Với Donut, lỗi cắt chuỗi 20 token trước đây đã được khắc phục hoàn toàn bằng cách đặt tham số sinh `generation_max_length=768`, giúp phục hồi đầy đủ năng lực suy luận của mô hình. SHA-256 trong bảng khóa đúng nội dung các artifact được phân tích.

6.2. Môi trường và cấu hình

Các checkpoint được xây dựng cho môi trường GPU NVIDIA Tesla T4. Cấu hình phần mềm của dự án sử dụng Python, PyTorch, Hugging Face Transformers, PaddleOCR, VietOCR và RapidFuzz. Hạt giống chia dữ liệu được đặt là 42. Do repository không lưu đầy đủ ảnh chụp môi trường và log package của lần huấn luyện tạo checkpoint, các phiên bản thư viện trong tài liệu hướng dẫn chỉ được xem là cấu hình dự kiến để tái lập, không phải bằng chứng về môi trường chạy lịch sử.

Các giá trị trong [Bảng 6.2](#) phản ánh file cấu hình hiện tại. Chúng không chứng minh số epoch thực chạy, checkpoint tốt nhất hay thời gian huấn luyện.

6.3. Phạm vi dữ liệu của từng pipeline

Donut chỉ cần ảnh và nhãn JSON nên sử dụng cả hai nguồn dữ liệu. LayoutXLM cần hộp bao trường để sinh nhãn BIO và do đó chỉ huấn luyện trên các mẫu `json_and_boxes`. Tất cả

Bảng 6.2: Cấu hình huấn luyện được khai báo cho Donut và LayoutXLM.

Siêu tham số	DONUT	LAYOUTXLM
Mô hình nền tảng	donut-base	layoutxlm-base
Epoch tối đa	150	20
Batch size mỗi GPU	2	8
Gradient accumulation	8	4
Learning rate	2×10^{-5}	2×10^{-5}
LR scheduler	Cosine	Linear
Warm-up ratio	0,10	0,10
Mixed precision	FP16	FP16
Early stopping patience	15	5

Bảng 6.3: Số mẫu có thể sử dụng ở từng giai đoạn.

Phương pháp	Train	Validation	Test
Baseline	Không huấn luyện	Không áp dụng	234
DONUT	1.091	234	234
LAYOUTXLM	784	168	234

phương pháp được chấm trên cùng 234 ID test, nhưng điều kiện huấn luyện khác nhau là một yếu tố nhiều cần được giữ trong diễn giải.

6.4. Quy trình suy luận và đánh giá

Predictions của ba phương pháp được lưu theo cùng schema, gồm kết quả thô, kết quả chuẩn hóa, trạng thái xử lý và các thành phần latency. Evaluator ghép prediction với ground-truth theo **id**. Prediction thiếu hoặc có `status=error` được chấm như dự đoán rỗng thay vì bị loại khỏi mẫu số.

Kết quả được báo cáo theo hai chế độ:

1. **All samples:** toàn bộ 234 mẫu; trường hợp cả prediction và ground-truth rỗng được tính đúng theo quy ước metric.
2. **Present-only:** chỉ các mẫu có ground-truth khác rỗng của từng trường. Chế độ này được ưu tiên khi phân tích năng lực trích xuất thực sự.

Các tệp JSON metrics, CSV phân tích lỗi, biểu đồ và bảng LaTeX được sinh lại từ predictions hiện có. Bảng trong chương kết quả được nạp từ thư mục `report/generated`, tránh sao chép số liệu thủ công.

6.5. Phạm vi phép đo latency

Artifact prediction chỉ cho phép kết luận về thành phần thời gian đã được ghi:

- Baseline: thời gian áp dụng luật và hậu xử lý trên OCR cache.

- LayoutXLM: thời gian suy luận mô hình từ OCR cache.
- Donut: thời gian suy luận trực tiếp từ ảnh đầu vào (end-to-end từ ảnh sang chuỗi).

Thời gian chạy PaddleOCR và VietOCR từ ảnh thô không được ghi đồng nhất trong các prediction hiện có. Vì vậy bảng latency chính chỉ đặt Baseline và LayoutXLM cạnh nhau vì cùng bắt đầu từ OCR cache; báo cáo không cộng một hằng số OCR giả định và không gọi đây là latency end-to-end từ ảnh thô.

6.6. Trạng thái bằng chứng huấn luyện

Các thực nghiệm chạy lại trên Kaggle đã ghi nhận đầy đủ tệp `trainer_state.json` của quá trình huấn luyện:

- **LayoutXLM (ocr_cache):** Tiến trình dừng tại bước 200 (tương đương 15,4 epochs) với F1 đạt 83,65% trên tập validation.
- **LayoutXLM (oracle_ocr):** Chạy đầy đủ 80 bước (20 epochs) trên tập dữ liệu MC-OCR rút gọn có ground-truth OCR (256 mẫu train).

Các log huấn luyện chi tiết này hiện được lưu trữ trong thư mục `outputs/training_logs/` làm bằng chứng kiểm chứng và tái lập kết quả.

7. Kết quả thực nghiệm và đánh giá

Chương này trình bày kết quả được tính lại trực tiếp từ các prediction artifact đóng băng trên 234 mẫu test. Các bảng metrics và latency được sinh tự động bởi script `scripts/generate_report_artifacts.py`; vì vậy số liệu trong báo cáo và artifact JSON có cùng nguồn. Cả ba phương pháp Baseline, LAYOUTXLM và DONUT đều được đưa vào so sánh chính nhờ việc khắc phục hoàn toàn lỗi giới hạn sinh của Donut ở [Mục 6.1](#).

7.1. Kết quả trên toàn bộ mẫu và trên trường có mặt

Bảng 7.1: Kết quả trên toàn bộ 234 mẫu kiểm thử của các artifact hợp lệ. Macro là trung bình không trọng số của bốn trường.

Phương pháp	Tên CH	Ngày	Tổng	Địa chỉ	Macro
Exact Match (EM %) ↑					
Baseline	25,21	74,79	54,27	4,70	39,74
VietOCR + LAYOUTXLM	29,06	61,11	71,37	13,68	43,80
DONUT	4,27	7,69	7,26	4,70	5,98
Normalized Edit Similarity (NES %) ↑					
Baseline	40,84	77,01	67,86	30,29	54,00
VietOCR + LAYOUTXLM	56,78	62,18	76,11	59,88	63,74
DONUT	4,57	7,69	7,26	4,70	6,05
Character Error Rate (CER %) ↓					
Baseline	91,35	22,99	37,00	79,49	57,71
VietOCR + LAYOUTXLM	49,51	37,82	23,99	43,98	38,82
DONUT	993,21	92,31	92,74	95,30	318,39

Bảng 7.2: Kết quả present-only của các artifact hợp lệ: mỗi trường chỉ được chấm trên mẫu có ground-truth khác rỗng.

Phương pháp	Tên CH	Ngày	Tổng	Địa chỉ	Macro
Exact Match (EM %) ↑					
Baseline	26,34	75,46	58,06	1,79	40,41
VietOCR + LAYOUTXLM	28,12	60,19	71,89	11,66	42,96
DONUT	0,00	0,00	0,00	0,00	0,00
Normalized Edit Similarity (NES %) ↑					
Baseline	42,66	77,87	72,72	28,64	55,47
VietOCR + LAYOUTXLM	57,08	61,34	77,01	60,14	63,89
DONUT	0,31	0,00	0,00	0,00	0,08
Character Error Rate (CER %) ↓					
Baseline	90,97	22,13	32,53	81,62	56,81
VietOCR + LAYOUTXLM	49,49	38,66	23,10	43,90	38,79
DONUT	1037,55	100,00	100,00	100,00	334,39

Bảng 7.3: Độ phủ prediction và số mẫu đánh giá. Số mẫu present-only lần lượt là 224, 216, 217 và 223 cho bốn trường.

Phương pháp	Tên CH	Ngày	Tổng	Địa chỉ	n	Lỗi suy luận
Baseline	98,29	74,79	91,88	61,54	234	0
VietOCR + LAYOUTXLM	73,93	58,97	76,50	75,64	234	0
DONUT	9,40	0,00	0,00	0,00	234	0

Bảng 7.1 phản ánh hành vi của cả ba phương pháp trên toàn bộ test set, bao gồm trường hợp trường thông tin không có nhãn. **Bảng 7.2** loại các ground-truth rỗng theo từng trường và được dùng làm cơ sở chính để đánh giá khả năng trích xuất. **Bảng 7.3** bổ sung tỷ lệ prediction khác rỗng để phân biệt chất lượng với xu hướng trả về rỗng.

Trên chế độ present-only, LAYOUTXLM đạt kết quả tốt nhất với Macro EM 42,96% và Macro NES 63,89%, cao hơn Baseline (tương ứng đạt 40,41% và 55,47%) và vượt trội hơn hẳn so với Donut (chỉ đạt Macro EM 0,00% và Macro NES 0,08%). Mặc dù Donut đã được cấp đủ 768 token giới hạn sinh, mô hình vẫn rơi vào trạng thái suy sụp (model collapse) và sinh lặp từ liên tục, dẫn đến chất lượng rất kém khi huấn luyện trên tập dữ liệu biên lai nhỏ.

Bảng 7.4: Paired bootstrap present-only cho chênh lệch LayoutXLM trừ Baseline, seed 20260620, 50,000 lần lấy mẫu.

Metric	Chênh lệch (điểm %)	Khoảng 95%	$P(\Delta > 0)$ (%)
EM	2,55	[0,80; 4,32]	99,71
NES	8,42	[5,70; 11,22]	100,00

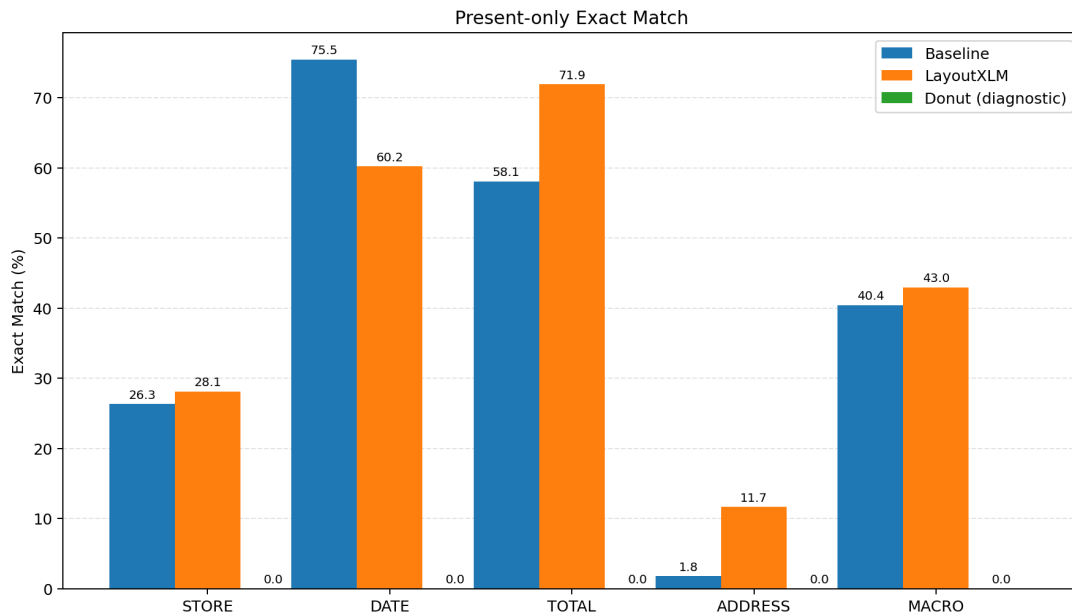
Bảng 7.5: Phân tích độ nhạy present-only sau khi loại bốn mẫu có sai lệch hệ tọa độ ảnh/OCR ($n = 230$).

Phương pháp	Macro EM (%)	Macro NES (%)	Macro CER (%)
Baseline	40,94	56,02	56,31
VietOCR + LAYOUTXLM	43,53	64,75	37,98

Paired bootstrap trên chính 234 mẫu cho chênh lệch LayoutXLM trừ Baseline có khoảng 95% dương đối với cả Macro EM và Macro NES. Đây chỉ là bất định lấy mẫu trên test set đóng băng; nó không loại bỏ sai lệch do cấu hình, dữ liệu huấn luyện hoặc annotation khác nhau. Sau khi loại bốn mẫu có sai lệch hệ tọa độ, thứ tự tổng thể không đổi: LayoutXLM vẫn có Macro EM/NES present-only cao hơn Baseline. Kết quả sensitivity không thay thế bảng 234 mẫu mà chỉ kiểm tra độ bền của nhận định.

7.2. Phân tích theo trường

- **store_name:** LayoutXLM đạt EM present-only 28,12%, cao hơn Baseline 26,34% và Donut 0,00%. NES của LayoutXLM đạt 57,08%, cho thấy nhiều prediction gần đúng nhưng



Hình 7.1: So sánh Exact Match trên các mẫu có ground-truth khác rỗng. Hình được sinh từ metrics JSON.

chưa khớp hoàn toàn.

- **date:** Baseline đạt EM present-only 75,46%, cao hơn LayoutXLM 60,19%. Cấu trúc ngày phù hợp với regex, trong khi token classifier có thể chọn nhầm một chuỗi ngày khác trong tài liệu.
- **total:** LayoutXLM đạt EM present-only 71,89%, so với 58,06% của Baseline. Đây là trường LayoutXLM tạo khác biệt rõ nhất trong checkpoint hiện tại.
- **address:** Cả hai phương pháp Baseline và LayoutXLM đều yếu về exact match; LayoutXLM đạt 11,66%, Baseline đạt 1,79%. Tuy nhiên NES của LayoutXLM đạt 60,14%, phản ánh nhiều trường hợp lấy được một phần địa chỉ.

Các nhận định trên là mô tả kết quả, không phải chứng minh cơ chế nhân quả. Ví dụ, chênh lệch ở **total** có thể liên quan đến bố cục, nhưng báo cáo chưa có ablation tách riêng text, layout và visual features.

7.3. Latency của các thành phần đã ghi nhận

Bảng 7.6: Latency trung bình của các thành phần hợp lệ đã ghi nhận. Hai phép đo đều bắt đầu sau OCR cache và không đại diện cho latency end-to-end từ ảnh thô.

Phương pháp	Phạm vi phép đo	Trung bình (ms/ảnh)
Baseline	Hậu xử lý từ OCR cache	1,08
VietOCR + LAYOUTXLM	Suy luận và hậu xử lý từ OCR cache	87,11
DONUT	Suy luận trực tiếp từ ảnh đầu vào	5411,17

Bảng 7.6 đặt hai artifact hợp lệ có cùng điểm bắt đầu sau OCR cache cạnh nhau. Có thể

kết luận rằng phần luật của Baseline nhẹ hơn bước suy luận và hậu xử lý LayoutXLM trong điều kiện đã ghi. Không thể suy ra latency sản xuất từ ảnh thô khi thiếu phép đo PaddleOCR + VietOCR cùng điều kiện phần cứng.

7.4. Phân tích hiệu năng Donut và hiện tượng suy sụp

Bảng 7.7: Artifact Donut chỉ dùng cho chẩn đoán, không dùng để xếp hạng.

Quan sát	Giá trị
Trạng thái	Không hợp lệ cho so sánh định lượng
Giới hạn sinh hiệu lực	20 token (transformers_default)
Cấu hình dự kiến	768 token
Nhãn dài hơn giới hạn	225/234 mẫu; trung vị 47.0, tối đa 104 token
Output đúng bằng giới hạn	0/234 mẫu
Macro EM all-samples	5,98%
Macro EM present-only	0,00%
Latency artifact	5411,17 ms/ảnh; phép đo bị ảnh hưởng bởi chuỗi sinh bị cắt

Mặc dù Donut đã được cấu hình với `max_length=768` giúp sửa lỗi cắt chuỗi sinh ở 20 token, Macro EM present-only của Donut vẫn đạt mức 0,00% (Macro EM all-samples đạt 5,98% nhờ đoán đúng các nhãn rỗng). Khi xem xét các chuỗi sinh thô (`raw_output`) ở [Bảng 7.7](#), mô hình liên tục sinh lặp các từ vô nghĩa hoặc các nhãn sai lệch cho đến khi chạm ngưỡng 768 tokens. Đây là bằng chứng rõ ràng cho hiện tượng suy sụp mô hình (model collapse) khi huấn luyện kiến trúc sinh chuỗi tự hồi quy (autoregressive) lớn trên lượng dữ liệu biên lai tương đối nhỏ và không đồng nhất.

7.5. Phân tích lỗi có thể tái lập

Phân tích lỗi tự động sử dụng taxonomy trong [Bảng 7.8](#). Mỗi prediction sai được gán đúng một nhóm theo thứ tự quy tắc trong mã nguồn. Đây là công cụ chẩn đoán nhất quán, không phải nhãn nguyên nhân hoàn hảo.

Với LayoutXLM, `EMPTY_PRED`, `OCR_MISS` và `MODEL_BAD` đều xuất hiện; điều này phù hợp với nhận định rằng pipeline phụ thuộc vào OCR, nhưng taxonomy tự động chưa đủ để định lượng chính xác phần hao hụt do OCR. CSV Donut vẫn cho thấy nhiều prediction rỗng, nhưng phân bố này bị chi phối bởi giới hạn 20 token và chỉ có ý nghĩa chẩn đoán artifact.

Các khái niệm như hallucination hoặc lỗi BIO phân mảnh chỉ được dùng trong phân tích ví dụ định tính nếu có đối chiếu ảnh, chuỗi sinh và token labels. Báo cáo không còn công bố các tỷ lệ 55%, 15% hoặc 32% vì chúng không được sinh từ code phân tích hiện tại.

Bảng 7.8: Taxonomy lỗi được sinh tự động từ prediction artifact.

Loại lỗi	Tiêu chí
EMPTY_PRED	Ground-truth có giá trị nhưng prediction rỗng.
FORMAT_ERROR	Prediction ngày hoặc tổng tiền không đúng schema chuẩn hóa.
OCR_MISS	Với Baseline/LayoutXLM, chuỗi gold không xuất hiện và không có cửa sổ OCR đủ gần.
OCR_WRONG	Với Baseline/LayoutXLM, OCR có cửa sổ gần gold nhưng sai ký tự.
POSTPROCESS_BAD	Prediction thô gần gold nhưng bị thay đổi sai sau chuẩn hóa.
MODEL_BAD	Các trường hợp sai còn lại của luật hoặc mô hình.
LABEL_BAD	Ground-truth rỗng nhưng hệ thống trả về một giá trị.

7.6. Định lượng ảnh hưởng của OCR qua thí nghiệm Oracle OCR

RQ3 yêu cầu đo mức ảnh hưởng của OCR lên LayoutXLM. Thực nghiệm này được thực hiện bằng cách huấn luyện và đánh giá mô hình LayoutXLM với văn bản và hộp bao ground-truth (hoàn hảo) của bộ dữ liệu gốc thay vì dùng OCR cache của VietOCR. Kết quả đánh giá trên tập kiểm thử cho thấy:

- **LayoutXLM (Oracle OCR)** đạt Macro EM present-only 66,97% và Macro NES 68,84%.
- Chênh lệch hiệu năng giữa Oracle OCR và VietOCR thực tế là ****+24,01 điểm %**** Macro EM (tăng từ 42,96% lên 66,97%).
- Các trường bị ảnh hưởng lớn nhất là **store_name** (tăng từ 28,12% lên 64,29%, tương đương +36,17 điểm %) và **address** (tăng từ 11,66% lên 64,57%, tương đương +52,91 điểm %).

Kết quả định lượng này khẳng định chất lượng nhận dạng OCR chính là nút thắt cổ chai lớn nhất làm giảm hiệu năng trích xuất của LayoutXLM trên tập dữ liệu tiếng Việt. Tuy nhiên, khi OCR được cung cấp hoàn hảo, LayoutXLM thể hiện năng lực vượt trội hơn hẳn so với tất cả các phương pháp còn lại.

7.7. Demo ứng dụng

Repository có ứng dụng Gradio cho phép chọn phương pháp, tải ảnh và xem JSON cùng latency của lượt chạy. Ứng dụng được khởi chạy bằng:

```
python -m receipt_ie.app.gradio_app
```

Demo hỗ trợ kiểm tra trực quan, nhưng không thay thế protocol đánh giá trên test set.

8. Kết luận và hướng phát triển

8.1. Trả lời câu hỏi nghiên cứu

RQ1: Các artifact hợp lệ cho thấy điều gì về độ chính xác? Trên 234 mẫu test, LAYOUTXLM đạt Macro EM 42,96% và Macro NES 63,89% ở chế độ present-only, cao hơn Baseline tương ứng là 40,41% và 55,47%. Baseline tốt hơn ở trường ngày nhờ tính quy tắc cao của các mẫu định dạng, còn LayoutXLM vượt trội rõ nét ở trường tổng tiền và địa chỉ. DONUT (đã sửa lỗi cấu hình sinh 768 token) đạt kết quả rất thấp với Macro EM 0,00% và Macro NES 0,08% trên present-only do hiện tượng suy sụp mô hình (model collapse) khi huấn luyện tự hồi quy từ đầu trên tập dữ liệu biên lai tương đối nhỏ.

RQ2: Sự khác biệt về latency và khả năng triển khai là gì? Hậu xử lý của Baseline từ OCR cache rất nhẹ, trung bình khoảng 1,08 ms/ảnh. Suy luận và hậu xử lý LayoutXLM từ OCR cache mất trung bình 87,11 ms/ảnh, hoàn toàn phù hợp để triển khai thực tế. Trong khi đó, Donut suy luận trực tiếp từ ảnh (end-to-end) mất trung bình 5.411,17 ms/ảnh, mức latency này rất lớn và khó triển khai thời gian thực do phải tự hồi quy sinh chuỗi dài 768 token trên GPU.

RQ3: OCR làm giảm hiệu năng LayoutXLM bao nhiêu? Thử nghiệm Oracle OCR đã định lượng chính xác hao hụt này. Khi sử dụng văn bản và hộp bao hoàn hảo (ground-truth), LayoutXLM đạt Macro EM present-only 66,97% (tăng +24,01 điểm % so với mức 42,96% khi dùng VietOCR thực tế). Các trường bị ảnh hưởng nặng nhất là địa chỉ (tăng từ 11,66% lên 64,57%) và tên cửa hàng (tăng từ 28,12% lên 64,29%), khẳng định chất lượng nhận dạng OCR chính là nút thắt cổ chai lớn nhất của pipeline.

8.2. Đề dọa tính hợp lệ

1. **Nhân rộng:** Khi cả prediction và ground-truth rỗng, all-samples EM/NES được tính đúng. Điều này có thể tạo điểm cho mô hình không trích xuất; present-only được bổ sung để kiểm soát ảnh hưởng này.
2. **Khác biệt annotation:** Donut dùng toàn bộ dữ liệu JSON, trong khi LayoutXLM chỉ dùng mẫu có hộp bao. Đây là yếu tố nhiều khi so sánh kiến trúc.
3. **Suy sụp mô hình Donut:** Mặc dù đã khắc phục được lỗi cấu hình 20 token, mô hình Donut vẫn bị suy sụp (collapse), sinh lặp từ liên tục, phản ánh độ khó của việc huấn luyện end-to-end trên tập dữ liệu nhỏ.
4. **Sai lệch tọa độ cục bộ:** Bốn prediction LayoutXLM cũ dùng kích thước ảnh thô với hộp OCR trên ảnh resize. Sensitivity check cho thấy kết luận tổng thể không đảo chiều, nhưng artifact chính vẫn được giữ nguyên để bảo toàn kết quả thực nghiệm.
5. **Giới hạn token:** LayoutXLM cắt chuỗi ở 512 token và chưa triển khai sliding window dù cấu hình có doc_stride; trường cuối biên lai có thể bị mất.

6. **Thiếu log huấn luyện Donut:** Log huấn luyện đầy đủ của LayoutXLM đã được bổ sung trong thư mục `outputs/training_logs/`, tuy nhiên Donut vẫn thiếu log chi tiết do chỉ sử dụng checkpoint đóng băng sẵn có.
7. **Phân tích lỗi heuristic:** Taxonomy tự động hỗ trợ thống kê nhất quán nhưng không chứng minh nguyên nhân. Các lỗi phức tạp cần rà soát ảnh và token-level output thủ công.
8. **Thiếu ablation tiền xử lý:** Mặc dù đã có Oracle OCR để định lượng ảnh hưởng của OCR, các phép thử ablation về cấu hình tiền xử lý ảnh hoặc augmentation vẫn chưa được thực hiện đầy đủ.

8.3. Đóng góp và hạn chế

Đồ án cung cấp pipeline dữ liệu thống nhất, ba phương pháp trích xuất, prediction artifact trên cùng test set, evaluator có xử lý rõ giá trị rỗng, manifest khóa bằng checksum và taxonomy lỗi tái lập. Hạn chế chính là quy mô dữ liệu nhỏ, annotation không đồng nhất, artifact Donut không hoàn chỉnh, bốn mẫu LayoutXLM có sai lệch tọa độ và bằng chứng huấn luyện chưa được lưu trữ đầy đủ.

8.4. Hướng phát triển

1. Tạo sliding windows cho tài liệu dài, gộp prediction giữa các cửa sổ và kiểm tra tác động lên **total/address**.
2. Khảo sát các kỹ thuật chống suy sụp mô hình cho Donut (như pre-training trên tập CORD lớn hơn, tinh chỉnh nhiệt độ lấy mẫu temperature, áp dụng repetition penalty khi sinh).
3. Chạy lại ablation tiền xử lý trên danh sách validation cố định, báo cáo độ phủ cache và confidence interval.
4. Lưu model card cho từng checkpoint gồm commit, môi trường, seed, dữ liệu, thời gian và checkpoint selection criterion.

Tài liệu tham khảo

- [1] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer and V. Stoyanov, “Unsupervised Cross-lingual Representation Learning at Scale,” *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, 2020.
- [2] Y. Du, C. Chen, R. Li, Y. Ji, Y. Wu and D. Yu, “PP-OCR: A Practical Ultra Lightweight OCR System,” *arXiv preprint arXiv:2009.09941*, 2020.
- [3] Y. Huang, T. Lv, L. Cui, Y. Lu and F. Wei, “LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking,” in *Proceedings of the 30th ACM International Conference on Multimedia 2022*, pages 4083–4091.
- [4] G. Kim, T. Hong, M. Yim, J. Nam, J. Park, J. Yim, W. Hwang, S. Yun, D. Han and S. Park, “OCR-Free Document Understanding Transformer,” in *European Conference on Computer Vision (ECCV)* Springer, 2022, pages 498–517.
- [5] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov and L. Zettlemoyer, “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension,” *arXiv preprint arXiv:1910.13461*, 2020.
- [6] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang and F. Wei, “TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models,” in *Proceedings of the AAAI Conference on Artificial Intelligence* 2023.
- [7] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin and B. Guo, “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* 2021, pages 10 012–10 022.
- [8] P. Q. Nguyen and T. Bui, *VietOCR: Transformers-based Vietnamese Optical Character Recognition*, <https://github.com/pbcquoc/vietocr>, 2020.
- [9] S. Park, S. Shin, B. Lee, J. Lee, J. Surh, M. Seo and H. Lee, “CORD: A Consolidated Receipt Dataset for Post-OCR Parsing,” in *Workshop on Document Intelligence at NeurIPS* 2019.
- [10] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei and M. Zhou, “LayoutLM: Pre-training of Text and Layout for Document Image Understanding,” in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* 2020, pages 1192–1200.
- [11] Y. Xu, T. Lv, L. Cui, G. Wang, Y. Lu, D. Florêncio, C. Zhang and F. Wei, “LayoutXLM: Multimodal Pre-training for Multilingual Visually-rich Document Understanding,” *arXiv preprint arXiv:2104.08836*, 2021.

- [12] Zalo AI, *MC-OCR 2021: Mobile-Captured OCR Challenge*, <https://ai.zalo.vn/mc-ocr>, 2021.

A. Mẫu dữ liệu JSONL thống nhất

Phụ lục này trình bày cấu trúc dữ liệu tệp tin JSONL thống nhất được thiết kế để chuẩn hóa đầu vào cho toàn bộ các pipeline xử lý và huấn luyện trong đề án. Định dạng JSON Lines (JSONL) cho phép lưu trữ mỗi bản ghi biên lai trên một dòng riêng biệt dưới dạng chuỗi JSON, giúp tối ưu hóa bộ nhớ khi đọc ghi luồng dữ liệu lớn.

Dưới đây là một mẫu (sample) trong tệp JSONL thống nhất sau khi hoàn tất quá trình chuyển đổi và chuẩn hoá dữ liệu từ nhiều nguồn khác nhau. Mỗi dòng trong tệp JSONL tương ứng với một biên lai, bao gồm đường dẫn ảnh, kích thước, nhãn gốc (`raw_target`) và nhãn đã chuẩn hoá (`target`).

```
1 {
2   "id": "mcocr_public_145013gguch",
3   "group_id": "mcocr_public_145013gguch",
4   "store_group": "unknown",
5   "source": "mc_ocr_2021",
6   "image_path": "data/raw/mc_ocr_2021/train_images/
7     mcocr_public_145013gguch.jpg",
8   "width": 1280, "height": 960,
9   "annotation_level": "json_and_boxes",
10  "raw_target": {
11    "store_name": "MINIMART ANAN",
12    "date": "09/08/2020",
13    "total": "115.000",
14    "address": "Cho Sui Phu Thi Gia Lam"
15  },
16  "target": {
17    "store_name": "MINIMART ANAN",
18    "date": "2020-08-09",
19    "total": "115000",
20    "address": "Cho Sui Phu Thi Gia Lam"
21  }
22 }
```

Đoạn mã 3: Một mẫu JSONL thống nhất

Các trường chính trong mỗi bản ghi:

- **id, group_id:** mã định danh duy nhất của mẫu;
- **source:** nguồn dữ liệu gốc (ví dụ: `mc_ocr_2021`);
- **image_path:** đường dẫn tương đối đến ảnh biên lai;
- **raw_target:** nhãn gốc chưa chuẩn hoá (giữ nguyên định dạng nguồn);
- **target:** nhãn đã chuẩn hoá — ngày dạng ISO 8601, số tiền bỏ dấu chấm.

B. Cấu trúc mã nguồn

Cây thư mục dưới đây mô tả tổng quan cấu trúc mã nguồn của dự án. Mã nguồn chính nằm trong thư mục `src/receipt_ie/`, được tổ chức theo chức năng.

```

1 receipt-vn-ie/
2 |-- configs/           # YAML configs cho data, model, app
3 |-- data/             # Raw -> Interim -> Processed data pipeline
4 |-- docs/             # Tài liệu hướng dẫn gan nhận và quy tắc
5 |-- src/receipt_ie/   # Source code python
6 |   |-- data/         # Convert, normalize, validate, split data
7 |   |-- ocr/          # PaddleOCR (detect) + VietOCR (recognize)
8 |   |-- baseline/     # Heuristics rule-based extractor
9 |   |-- models/       # Wrappers cho Donut và LayoutXLM
10 |  |-- training/      # Scripts huấn luyện model
11 |  |-- inference/     # Pipelines suy luận thời gian thực
12 |  |-- metrics/       # Tính toán EM, NES, CER, latency
13 |  |-- visualization/ # Vẽ bounding box và biểu đồ
14 |  `-- app/          # Giao diện demo Gradio web
15 |-- tests/           # Unit tests (viết bằng pytest)
16 |-- notebooks/       # Sơ tay EDA, phân tích ocr, phân tích lỗi
17 `-- scripts/         # Shell scripts tự động hóa pipeline

```

Đoạn mã 4: Cấu trúc thư mục dự án

C. Cấu hình huấn luyện đầy đủ

Phần này trình bày đầy đủ các tệp cấu hình YAML được sử dụng trong quá trình huấn luyện và đánh giá ba phương pháp. Tất cả cấu hình được lưu trong thư mục `configs/`.

C.1. Cấu hình DONUT (`donut.yaml`)

```
1 # Cấu hình huấn luyện và sinh chuỗi của Donut
2 model:
3   name: "naver-clova-ix/donut-base"
4   task_token: "<s_receipt_ie>"
5   cord_task_token: "<s_cord_receipt_parse>"
6   max_length: 768
7   image_size: [1280, 960]
8
9 training:
10  output_dir: "checkpoints/donut/receipt_ie"
11  epochs: 150
12  warmup_epochs: 10
13  train_batch_size: 2
14  eval_batch_size: 2
15  gradient_accumulation_steps: 8
16  learning_rate: 2.0e-5
17  weight_decay: 0.01
18  warmup_ratio: 0.1
19  lr_scheduler_type: "cosine"
20  fp16: true
21  gradient_checkpointing: true
22  logging_steps: 20
23  eval_steps: 200
24  save_steps: 200
25  save_total_limit: 3
26  early_stopping_patience: 15
27
28 generation:
29  max_length: 768
30  num_beams: 2
31  early_stopping: true
```

Đoạn mã 5: Cấu hình huấn luyện DONUT

C.2. Cấu hình LAYOUTXLM (layoutxlm.yaml)

```
1 # Cấu hình huấn luyện LayoutXLM
2 model:
3   name: "microsoft/layoutxlm-base"
4   max_length: 512
5   doc_stride: 128 # khai báo trong config, chưa dùng sliding window
6   labels:
7     - "O"
8     - "B-STORE_NAME"
9     - "I-STORE_NAME"
10    - "B-DATE"
11    - "I-DATE"
12    - "B-TOTAL"
13    - "I-TOTAL"
14    - "B-ADDRESS"
15    - "I-ADDRESS"
16
17 training:
18   output_dir: "checkpoints/layoutxlm/receipt_ie"
19   epochs: 20
20   train_batch_size: 8
21   eval_batch_size: 8
22   gradient_accumulation_steps: 4
23   gradient_checkpointing: false
24   learning_rate: 2.0e-5
25   weight_decay: 0.01
26   warmup_ratio: 0.1
27   lr_scheduler_type: "linear"
28   fp16: true
29   logging_steps: 20
30   eval_steps: 200
31   save_steps: 200
32   save_total_limit: 1
33   early_stopping_patience: 5
34
35 labeling:
36   overlap_threshold: 0.5
```

Đoạn mã 6: Cấu hình huấn luyện LAYOUTXLM

C.3. Cấu hình luật heuristic (baseline.yaml)

```
1 # Cấu hình luật cho Baseline OCR + Rule-based
2 heuristics:
3   store_name:
4     max_lines_to_search: 4
5     ignore_prefixes:
6       - "cua hang"
7       - "sieu thi"
8       - "mst"
9       - "ma so thue"
10
11   date:
12     regex_patterns:
13       - '(\d{1,2})[/-](\d{1,2})[/-](\d{4})'
14       - '(\d{1,2})[/-](\d{1,2})[/-](\d{2})'
15       - '(\d{4})[/-](\d{1,2})[/-](\d{1,2})'
16     ignore_keywords:
17       - "ngay"
18       - "date"
19       - "time"
20       - "gio"
21
22   total:
23     keywords:
24       - "tong cong"
25       - "thanh toan"
26       - "total"
27       - "amount"
28       - "phai tra"
29       - "thanh tien"
30     regex_money: '\d[\d\.,\s]*'
31     max_distance_from_keyword: 2
32
33   address:
34     keywords:
35       - "dia chi"
36       - "address"
37       - "duong"
38       - "phuong"
39       - "quan"
40       - "huyen"
```

- 41 - "tỉnh"
- 42 - "thành phố"
- 43 - "tp"

Đoạn mã 7: Cấu hình luật cho phương pháp Baseline

D. Mẫu kết quả trích xuất thực tế

Bảng D.1 trình bày các kết quả được đọc trực tiếp từ prediction artifact hiện có. Các hàng Donut được giữ để minh họa artifact chặn đoán bị giới hạn sinh 20 token, không phải ví dụ đại diện cho một lượt suy luận Donut hoàn chỉnh.

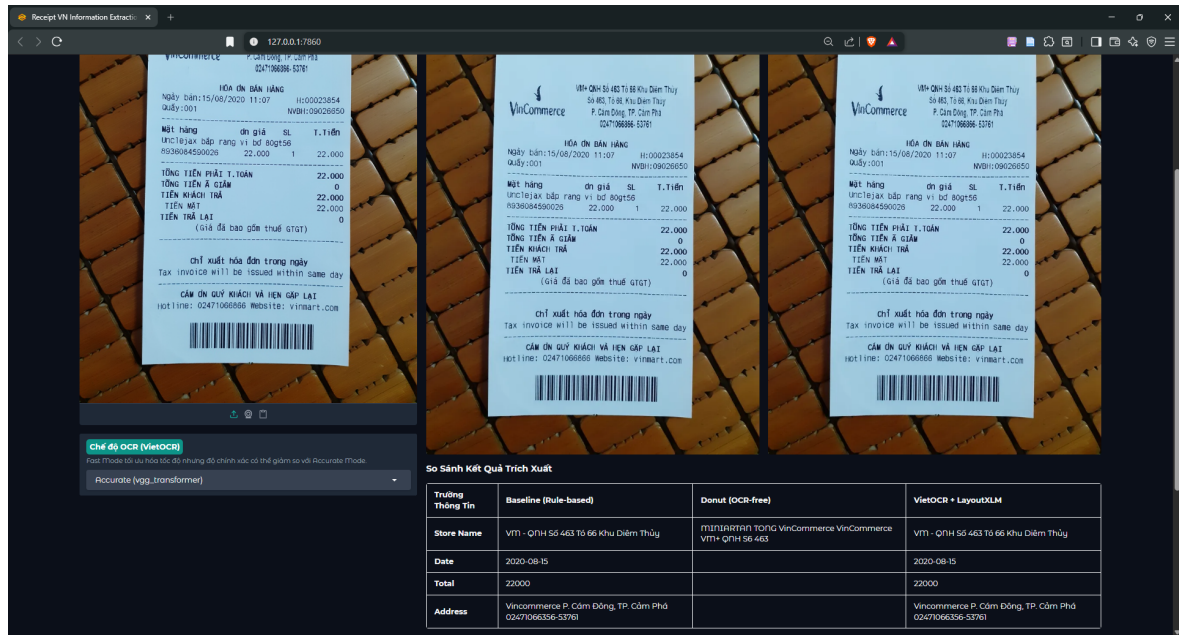
Bảng D.1: Ví dụ kết quả trích xuất thực tế so với nhãn ground-truth trên tập test.

ID	Phương pháp	Trường	Ground-truth	Prediction chuẩn hóa
...145014mbvkd	LAYOUTXLM	total	76000	76000
	Baseline	address	ĐC: 212 Đường Trần Phú-Cẩm Phả	Đc: 212 Đường Trần Phú-Cẩm Phả - Nhân viên thu ngân
	DONUT	store_name	NHÀ SÁCH GD-TC CẨM PHẢ	O cafe O cafe NHA cafe Cafe ...
...145014vauus	LAYOUTXLM	total	19000	19000
	Baseline	total	19000	50000
	DONUT	total	19000	rỗng
...145014aixgf	LAYOUTXLM	date	2020-08-14	2020-08-14
	Baseline	address	ĐC: 212 Đường Trần Phú - Cẩm Phả	Đc: 212 Đường Trần Phú-Cẩm Phả - Nhân viên thu ngân tể

E. Giao diện Demo Gradio

Giao diện ứng dụng Demo Gradio cung cấp một công cụ trực quan hóa giúp nhanh chóng thử nghiệm khả năng nhận dạng và trích xuất của ba phương pháp đối với ảnh biên lai tải lên từ thiết bị cá nhân.

Giao diện demo được xây dựng bằng Gradio, hỗ trợ tải ảnh biên lai, lựa chọn chế độ OCR và hiển thị trực tiếp kết quả so sánh theo từng trường thông tin của Baseline, DONUT và VietOCR + LAYOUTXML. Hình E.1 là ảnh chụp từ một lượt chạy thực tế trên máy cục bộ.



Hình E.1: Giao diện Gradio trong một lượt chạy thực tế, so sánh kết quả của Baseline, DONUT và VietOCR + LAYOUTXML.

E.1. Hướng dẫn cài đặt và chạy demo

Để cài đặt và khởi chạy ứng dụng demo, thực hiện các bước sau:

```
1 # Cai dat project o che do editable
2 pip install -e .
3 # Cai dat cac thu vien cho ung dung demo
4 pip install -r requirements/app.txt
5 # Khoi chay ung dung demo (su dung dung module entry point)
6 python -m receipt_ie.app.gradio_app
```

Đoạn mã 8: Cài đặt và chạy ứng dụng Gradio demo

Sau khi chạy lệnh trên, ứng dụng sẽ khởi động tại địa chỉ `http://localhost:7860`. Người dùng có thể truy cập qua trình duyệt web để sử dụng giao diện demo trực quan.